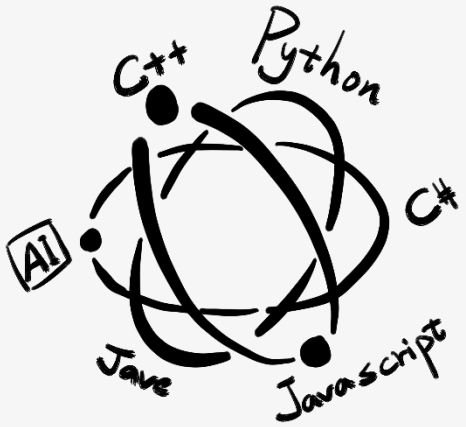




树与图

酷哥创客 COOLMAKER







1 温故知新

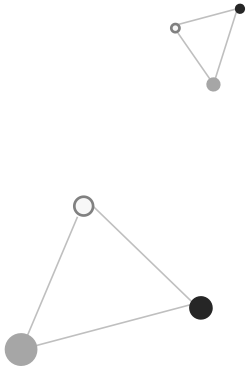
2 知识宝典

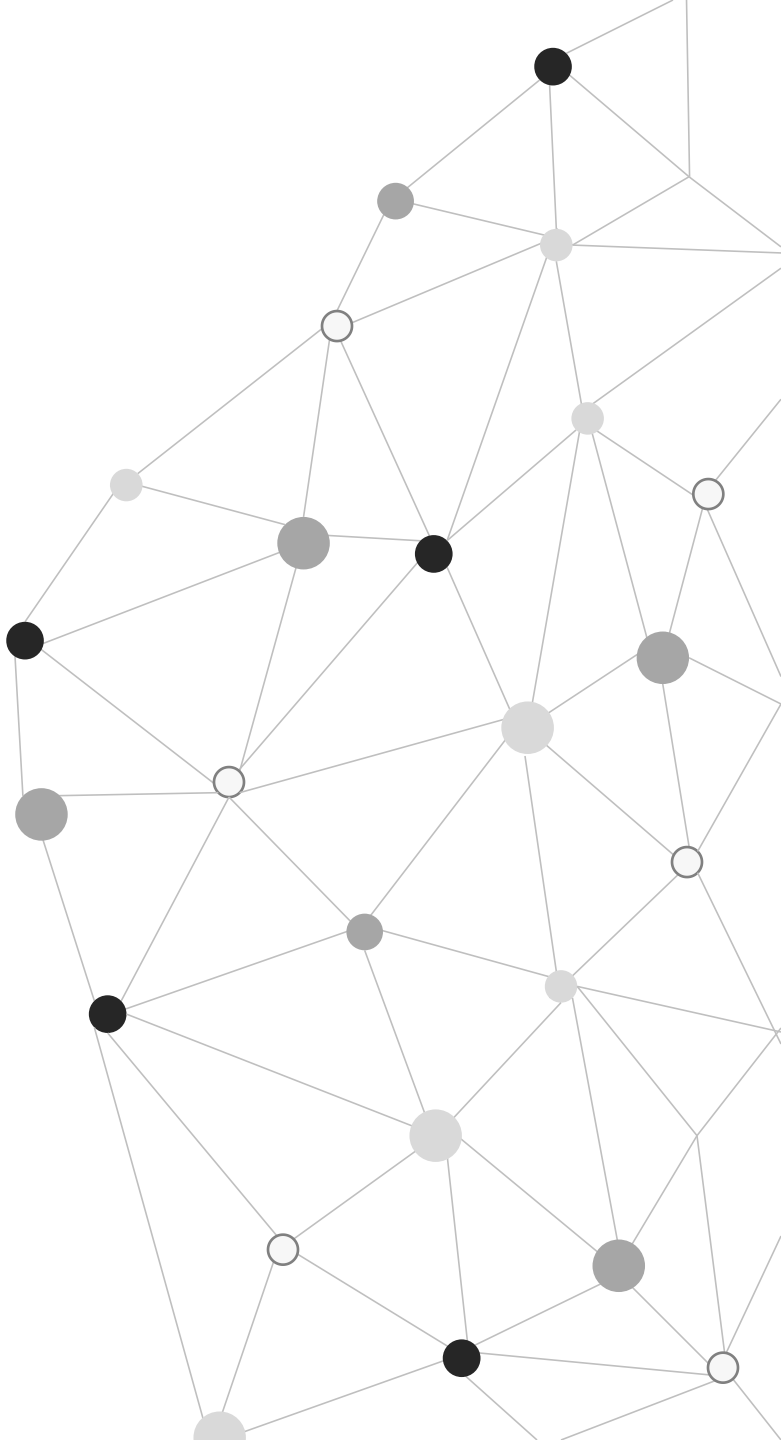
3 运用自如

4 回顾总结



知 识 宝 典







COOLMAKER
酷哥创客 人工智能教育



第一课





CSP-S 2025提高组复赛一二三等奖分数和人数

-S提高组	一等奖			二等奖			三等奖			一等
省份	2023	2024	2025	2023	2024	2025	2023	2024	2025	获奖率
重庆	200	250	217	80	105	60	55	60	30	25.2%
浙江	155	200	166	80	105	60	55	60	30	23.7%
广东	150	180	152	80	105	60	55	60	30	21.3%
山东	140	180	151	80	105	60	55	60	30	23.3%
上海	140	175	151	80	105	60	55	60	30	21.6%
四川	155	180	145	80	105	60	55	60	30	21.5%
北京	150	175	145	80	105	60	55	60	30	21.1%
湖南	140	165	140	80	105	60	55	60	30	20.8%
香港	175	175	137	80	105	60	55	60	30	19
江苏	150	170	132	80	105	60	55	60	30	21.7%
福建	140	165	132	80	105	60	55	60	30	19.8%
安徽	140	165	115	80	60	30	55			20.6%
澳门	105	165	113	60	60	30				13
江西	130	145	104	55	60	30				20.7%
河南	115	130	100	55	60	30				14.3%
辽宁	105	125	100	55	60	30				22.3%
湖北	115	125	100	55	60	30				20.0%
陕西	115	130	100	55	60	30				19.0%
山西	105	125	88	55	60	30				100
吉林	125	125	80	55	60	30				17.8%
天津	105	125	80	55	60	30				16.2%
广西	105	125	76	55	60	30				17.3%
河北	125	125	69	55	60	30				16.5%
黑龙江	125	125	60	55	60	30				80
云南	60	105	60	55	60	30				15.3%
海南	85	105	60	55	60	30				14.1%
新疆	85	120	60	55	60	30				68
贵州	85	105	60	55	60	30				13.3%
宁夏	60	120	60	55	60	30				10
内蒙古	60	105	60	55	60	30				8.8%
甘肃	90	105	60	60	60	30				11
青海		65	60		60	30				0
总计		20.0%								20.31%

COOLMAKER 酷哥创客 人工智能教育

CSP-S2 提高组复赛考什么?

四道题　　100分/题，共400分

考试时间:　　4小时

考试策略:　T1争取满分　　T2尽可能多的分数

T3尽可能多的分数　T4尝试暴力骗分

COOLMAKER
酷哥创客 人工智能教育

T1 考点：普及算法和数据结构-加深

第一题	2025	2024	2023	2022	2021	2020
题目名称	社团招新(黄)	决斗(橙)	密码锁(橙)	假期计划(蓝)	廊桥分配(绿)	儒略日(绿)
考察知识点	反悔贪心	贪心、双指针、Dilworth定理	枚举	最短路、BFS、枚举	优先队列、模拟、枚举	模拟、二分、数学

T2 考点：提高算法与数据结构

第二题	2025	2024	2023	2022	2021	2020
题目名称	道路修复(青)	超速检查(绿)	消消乐(蓝)	策略游戏(绿)	括号序列(蓝)	动物园(黄)
考察知识点	最小生成树	二分、贪心、数学、模拟	动态规划、序列哈希	贪心、RMQ(ST表、线段树)	多维区间DP	数学、贪心

T3 考点：提高算法与数据结构

第三题	2025	2024	2023	2022	2021	2020
题目名称	谐音替换(蓝)	染色(蓝)	结构体(蓝)	星战(紫)	回文(绿)	函数调用(蓝)
考察知识点	字典树、扫描线、线段树、AC自动机	动态规划优化	大模拟	基环树、集合哈希	构造、贪心、模拟	动态规划、数学、拓扑排序

T4 考点：提高、NOI级算法与数据结构

第四题	2025	2024	2023	2022	2021	2020
题目名称	员工招聘(紫)	擂台游戏(黑)	种树(蓝)	数据传输(紫)	交通规划(紫)	贪吃蛇(黑)
考察知识点	动态规划优化、组合数学、容斥	DFS、差分、数据结构(线段树)	贪心、二分	DDP(动态DP)、矩阵乘法、带权并查集、LCA(倍增、树链剖分)	网络流、对偶图、最短路、区间DP	贪心、双端队列、模拟

提高组复赛的关键技巧

- 赛前准备：夯实基础，减少**临场失误**
- 赛中策略：合理分配时间，**最大化得分**
- 核心算法技巧：针对性突破**高频考点**

提高组复赛的核心是 “基础扎实 + **策略灵活** + 细节严谨” 。赛前通过系统训练**夯实算法基础**，赛中按 “**得分效率**” 分配时间，**优先拿稳易题和部分分**，编码时注重模块化和调试技巧，赛后及时复盘错题。记住：竞赛不仅是比谁会的算法多，更是比谁能在**有限时间内拿到更多分数。**

复赛备赛要点

COOLMAKER
酷哥创客 人工智能教育

- 赛前立目标、打基础、稳固根基，找出自己的薄弱点，制定复习备考计划
- 赛前熟悉考试OI赛制模式及考场环境。编程平台：Dev-C++（Windows），Code::Blocks（NOI Linux）
- 赛中防止低级失误而爆零，如头文件、文件操作、四名、格式、及时保存等 。
- 掌握好构造数据、本地断点或打印调试技巧，争一保二。
- 遇到不会的题，冷静分析逐步分解，拿部分分，基础题不失分。
- 合理安排时间，尽量避免无法完成考试。
- 保持好心态，遵守考场纪律，顺利完成考试。



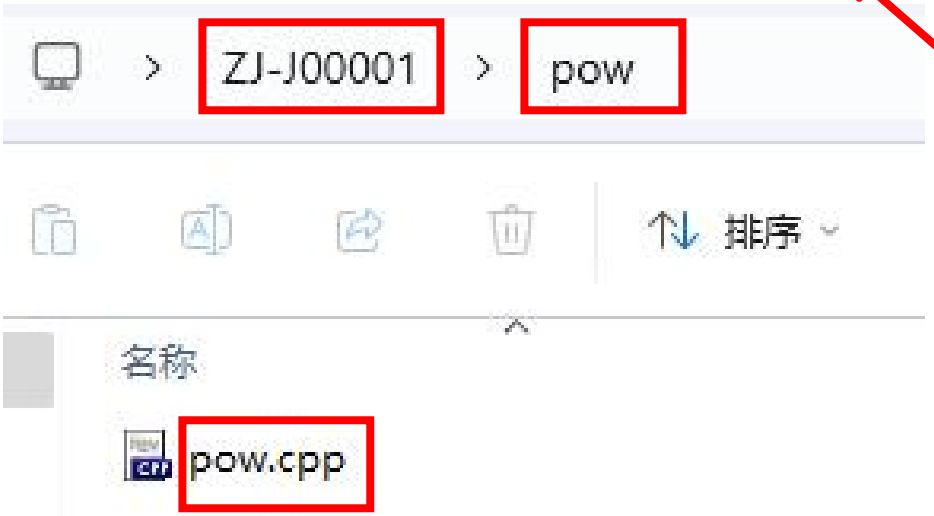
1、文件夹建立错误


题目名称
乘
张三
pow
名称
pow.cpp
ZJ-J00001
a
排序
名称
pow.cpp
512 MiB
10
20
上升点列
传统型
point
point
point.in
ZJ-J00001
pow
排序
名称
a.cpp

正确的文件夹建立

文件目录结构：

- 根目录是准考证号
- 子目录是程序名
- 子目录里再放程序名.cpp




```
ZJ-J00166
├─decode
│      decode.cpp
├─expr
│      expr.cpp
├─point
│      point.cpp
└─pow
       pow.cpp
```

2、没有加文件操作

```
int main()
{
    int n; cin >> n;
    for (int i = 1; i
        cout << i <<

    return 0;
}
```

```
int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; i ++)
        cout << i << "\n";

    freopen("pow.in", "r", stdin);
    freopen("pow.out", "w", stdout);

    return 0;
}
```

```
"pow1.in", "r", stdin);
"pow.out", "w", stdout);

in >> n;
 i = 1; i <= n; i ++)
 << i << "\n";

;
```



正确的文件操作

COOLMAKER
酷哥创客 人工智能教育

在主函数的**开头**写打开文件操作代码：

freopen("程序名.in", "r", stdin) ;
freopen("程序名.out", "w", stdout) ;

```
int main()
{
    freopen("pow.in", "r", stdin);
    freopen("pow.out", "w", stdout);

    int n; cin >> n;
    for (int i = 1; i <= n; i ++)
        cout << i << "\n";

    return 0;
}
```



4644: 决斗（duel）

COOLMAKER
酷哥创客 人工智能教育

题目描述

今天是小 Q 的生日，他得到了 n 张卡牌作为礼物。这些卡牌属于火爆的“决斗怪兽”，其中，第 i 张卡代表一只攻击力为 r_i ，防御力也为 r_i 的怪兽。

一场游戏分为若干回合。每回合，小 Q 会选择某只怪兽 i 以及**另一只**怪兽 $j(i \neq j)$ ，并让怪兽 i 向怪兽 j 发起攻击。此时，若怪兽 i 的攻击力小于等于怪兽 j 的防御力，则无事发生；否则，怪兽 j 的防御被打破，怪兽 j 退出游戏不再参与到剩下的游戏中。一只怪兽在整场游戏中**至多**只能发起一次攻击。当未退出游戏的怪兽都已发起过攻击时，游戏结束。

小 Q 希望决定一组攻击顺序，使得在游戏结束时，未退出游戏的怪兽数量尽可能少。

输入

输入的第一行包含一个正整数 n ，表示卡牌的个数。

输入的第二行包含 n 个正整数，其中第 i 个正整数表示第 i 个怪兽的攻击力及防御力 r_i 。

输出

输出一行包含一个整数表示游戏结束时未退出游戏的怪兽数量的最小值。

样例输入1 复制

```
5
1 2 3 1 2
```

样例输出1 复制

```
2
```

【数据范围】

对于所有测试数据，保证：$1 \leq n \leq 10^5$ ，$1 \leq r_i \leq 10^5$ 。

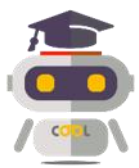
测试点	n	r_i	特殊性质
$1 \sim 4$	≤ 10	$\leq 10^5$	无特殊性质
$5 \sim 10$	$\leq 10^5$	≤ 2	
$11 \sim 15$	≤ 30	$\leq 10^5$	特殊性质 A
$16 \sim 20$	$\leq 10^5$		无特殊性质

特殊性质 A：保证每个 r_i 在可能的值域中独立均匀随机生成。

25 分 （测试点 11~15 ， $n \leq 30, r \leq 10^5$, 保证每个 r_i 在可能的值域中独立均匀随机生成。）

由于 n 只有30，而 r_i 值域很大，且数据随机，因此极大概率 r_i 互不相同，此时考虑贪心，排序使得 $r_1 < r_2 < r_3 < r_4 < \cdots < r_n$ 时，可以依次让每个 r_i 攻击 r_{i-1} ，此时只会剩下攻击力最大的怪兽，即答案为 1 。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main()
{
    cout<<1;
    return 0;
}
```

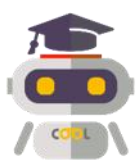


30 分 （测试点 5~10 ， $r_i \leq 2$）

.MAKER
人工智能教育

此时的 r_i 的值只有 1 或者 2 两种，统计 1 的个数 x , 统计 2 的个数 y，此时答案就是 $max(x,y)$ ，因为 我们可以让 $min(x,y)$ 个 2 去攻击 1 , 此时剩下 $max(x,y)$ 个怪兽 或者说 剩下 $n-min(x,y)$ 个怪兽。（因为 $max(x,y)+min(x,y)=x+y=n$）

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N=1e5+5;
int n, s[N];
int main()
{
    cin>>n;
    for(int i=0;i<n;i++){
        int r;
        cin>>r;
        s[r]++;
    }
    cout<<max(s[1],s[2]);
    return 0;
}
```



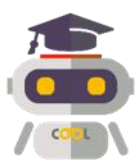
100 分 解法1 O(nlogn)

COOLMAKER
酷哥创客 人工智能教育

类似田忌赛马的贪心，我们要尽可能让每个怪兽充分发挥作用，即攻击一个怪兽后再被另一个怪兽攻击，尽量干掉一个比它攻击值小的怪兽，可以发现攻击值越大的应该越晚被攻击。

那么可以有**贪心策略**，将怪兽按照**攻击力从小到大排序**，从小到大枚举怪兽，维护两个指针， 指针 i 代表当前还**未被攻击**的攻击力最小怪兽，指针 j 代表当前还**未攻击**的攻击力最小怪兽，如果 $r_i < r_j$ 那么 $i = i + 1$, $j = j + 1$, 否则 $i = i + 1$, 最终 $[i, n]$ 范围的怪兽就是未退出游戏的怪兽数量，输出 $n - i + 1$。

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N=1e5+5;
int n,r[N];
int main()
{
    cin>>n;
    for(int i=1;i<=n;i++)cin>>r[i];
    sort(r+1,r+n+1);
    int i=1;
    for(int j=2;j<=n;j++)
    {
        if(r[i]<r[j])i++;
    }
    cout<<n-i+1;
    return 0;
}
```



100 分 解法2 O(n)

COOLMAKER
酷哥创客 人工智能教育

按攻击力从大到小排序，类似导弹拦截所需最少拦截系统的思路。

形式化地考虑这个问题。若 i 成功攻击了 j，我们连一条 $i \to j$ 的边。可以发现：

- 每个点的出度至多为 1。这是因为一只怪兽在整场游戏中至多只能发起一次攻击。
- 每个点的入度至多为 1。这是因为一只怪兽被成功攻击了就不再参与到剩下的游戏中。
- 图中不存在环。这是因为如果 i 成功攻击 j，必须有 $r_i > r_j$，而 $>$ 是严格弱序关系，不存在环。

这说明**图由若干链构成**。更进一步的，如果能连出这样的若干链，则一定可以达成（每条链按照顺序攻击即可）。而每条链最终会恰好留下一个点。所以，我们要找的是，所有的怪兽，**在 r 的严格小于关系下，至少要拆成多少条链才能覆盖所有**。

具体的，我们定义 (i, r_i) 的偏序关系为 $(i, r_i) \leq (j, r_j)$ 当且仅当 $r_i < r_j$ 或 $i = j$。显然这是一个偏序关系。我们要找到它的一个最小链覆盖。

根据 **Dilworth 定理**，最小链覆盖等于最长反链，也就是最多的两两不偏序的怪兽个数。即在**从小到大排序的序列中找最长不下降子序列**，此时最长不下降子序列（两两不偏序）即要求所有 r 相等，因此统计 r 中**众数的出现次数**即可。

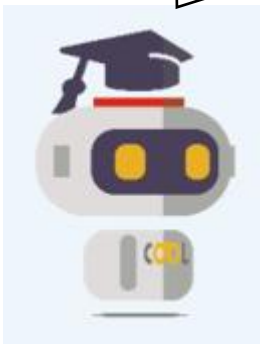
```cpp
#include <bits/stdc++.h>
using namespace std;
const int N=1e5+5;
int n,r,s[N],ans=0;
int main()
{
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>r;
        ans=max(ans,++s[r]);
    }
    cout<<ans;
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

网址：

oj.coolguymaker.com

请你跟我一起进入
做题的平台吧







用户名： 购买课程的手机号

密码 ： 默认为123456


COO MAKER
酷哥创客 人工智能教育
余博士教编程OJ 酷哥OJ
主页
选择题
编程题
作业
竞赛
题单
班级
分类
状态
排名
登录好以后
选择右上角的班级
还有另一个自己在这里等你
也许另一个人也和你一样，发现着自己的创新和诚心
练习
技术支持: hh@coolguymaker.com

2025CSP初赛冲刺暑...
10/04 CSP-J初赛模拟-试卷7
100%
下次直播时间:
授课老师：方培辉

2025C++GESP考级...
10/04 GESP冲刺3-4级编程题详解
50%
下次直播时间：2025-09-14 13:30
授课老师：应程杰

编程与数学基础班级
10/04 第01课_认识数
5%
下次直播时间:
授课老师：余昌盛

2025C++秋季二期晚...
10/04 第02课_整型变量与输入
3%
下次直播时间：2025-09-27 19:00
授课老师：柳徐吉

2025C++秋季一期上...
10/04 第04课_拆位运算
6%
下次直播时间：2025-10-02 10:00
授课老师：应程杰

2025CSP-S复赛冲刺...
10/04 复赛冲刺第01课_树和图专题
8%
下次直播时间：2025-10-04 10:00
授课老师：张谦

2025C++高级算法秋...
10/04 第04课_表达式求值
8%
下次直播时间：2025-10-24 19:00
授课老师：方培辉

2025CSP-J复赛冲刺...
10/04 复赛冲刺第03课_递归和搜索算...
25%
下次直播时间：2025-10-02 19:00
授课老师：方培辉

2025CSP-J复赛冲刺...
10/04 复赛冲刺第01课_STL容器与模拟
8%
下次直播时间：2025-10-04 10:00
授课老师：柳徐吉

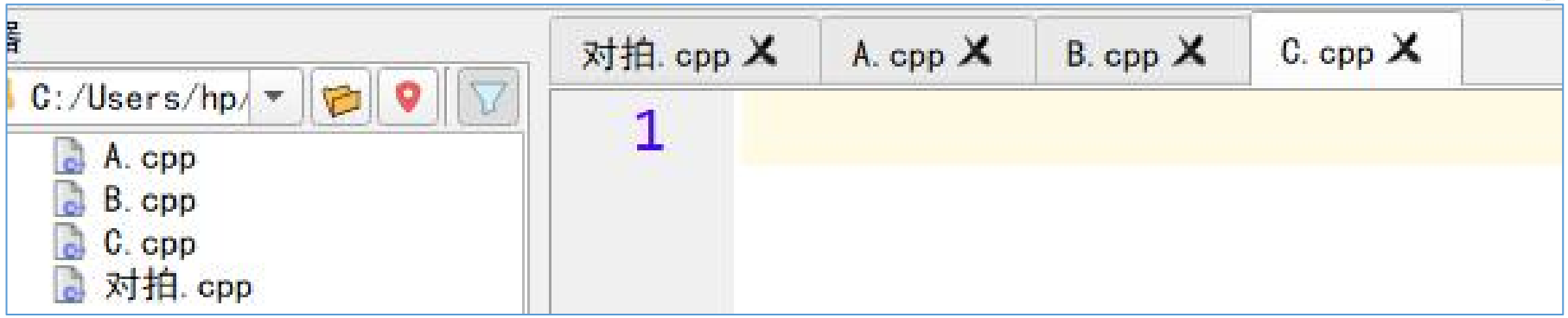
找到自己所对应的班级



对拍







准备4个文件

对拍.cpp 用于测试代码

A.cpp 用于生成数据

B.cpp 保证答案正确的暴力代码

C.cpp 被测试的代码

对拍.cpp | A.cpp[*] | B.cpp | C.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
mt19937_64 rnd(time(0)); //随机数生成器
//生成 l到 r 范围内的数
long long range(long long l, long long r)
{
    return rnd() % (r - l + 1) + l;
}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n = range(2, 5);
    cout << n << "\n";
    for(int i = 1; i <= n; i++)
    {
        cout<<range(1,100)<<" ";
    }
    return 0;
}
```

A.cpp 用于生成数据

mt19937_64 rnd(time(0));
是 64 位的**梅森旋转随机数生成器**，其生成的随机数范围为 无符号 64 位整数的完整区间，即 0 到 $2^{64}-1$（约 1.8×10^{19}）。

range(l,r)
通过取余来获得 l 到 r 范围内的数。

对拍.cpp | A.cpp | B.cpp[*] | C.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x;
        cin >> x;
        cout << x << " ";
    }
    return 0;
}
```

B.cpp 保证答案正确的暴力代码

假如题目是让你输出输入的这 n 个数，代码就如右图所示。



对拍.cpp | A.cpp | B.cpp[*] | C.cpp[*]

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n;
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        int x;
        cin>>x;
        cout<<1<<" ";
    }
    return 0;
}
```

C.cpp 被测试的代码



对拍.cpp A.cpp B.cpp C.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() //对拍
{
    while(1)
    {
        system("A.exe > 1.in");//随机
        system("B.exe < 1.in > 1.ans");//保证答案正确的暴力代码
        system("C.exe < 1.in > 1.out");//被拍的代码
        //错误信息可以在 1.log 找到,
        //错误测试点的输入输出和答案在对应的 .in / .out / .ans 文件中。
        if(system("fc 1.ans 1.out >1.log"))
        {
            cout << "error" << endl;
            return 0;
        }
        else cout << "OK" << endl;
    }
    return 0;
}
```

对拍.cpp 用于测试代码

确保A.cpp B.cpp C.cpp 三个代码都运行生成过 .exe 文件。

此时点击运行 对拍.cpp 代码，会将A.exe中的输出写入到 1.in中。

B.exe 会读入 1.in 文件中的数据，并将输出结果写入到 1.ans 中

C.exe 会读入 1.in 文件中的数据，并将输出结果写入到 1.ans 中

system("fc 1.ans 1.out >1.log") 会比较 1.ans 和 1.out 中输出，如果有错误会返回 true



C:\Users\hp\Desktop\democpp\对拍\对拍.exe

```
error

--------------------------------
Process exited after 0.4396 se

Press ANY key to exit.
```

demo

1.in - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
2
69 97
```

1.out - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
1 1
```

1.ans - 记事本

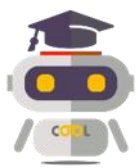
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
69 97
```

1.log - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
正在比较文件 1.ans 和 1.OUT
***** 1.ans
69 97
***** 1.OUT
1 1
*****
```



7.图论算法

- 【6】最小生成树：Prim和Kruskal等算法
- 【6】单源最短路：Bellman-Ford、Dijkstra、SPFA等算法
- 【7】单源次短路
- 【6】Floyd-Warshall算法
- 【6】有向无环图的拓扑排序
- 【6】欧拉道路和欧拉回路
- 【6】二分图的判定
- 【7】强连通分量
- 【7】割点、割边
- 【6】树的重心、直径、DFS序与欧拉序
- 【6】树上差分、子树和与倍增
- 【6】最近公共祖先

COOLMAKER
酷哥创客 人工智能教育



图





最短路

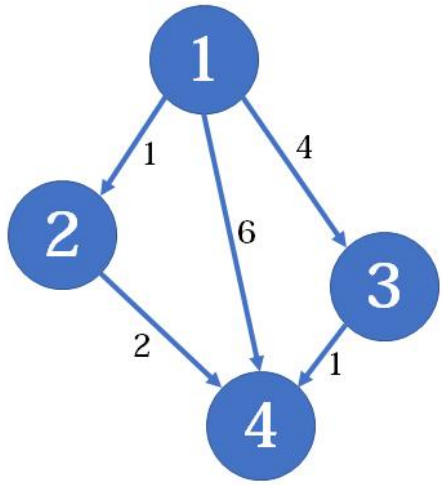
COOLMAKER
酷哥创客 人工智能教育

最短路问题（Short-path Problem）是网络理论解决的典型问题之一，基本内容是：若网络中的每条边都有一个数值（长度、成本、时间等），则找出两节点（通常是源节点和阱节点）之间总权和最小的路径就是最短路问题。



最短路径问题

最短路问题。如下图，我们想知道，某点到某点最短的路径有多长？



图中点1到点4的最短路径长度应为3

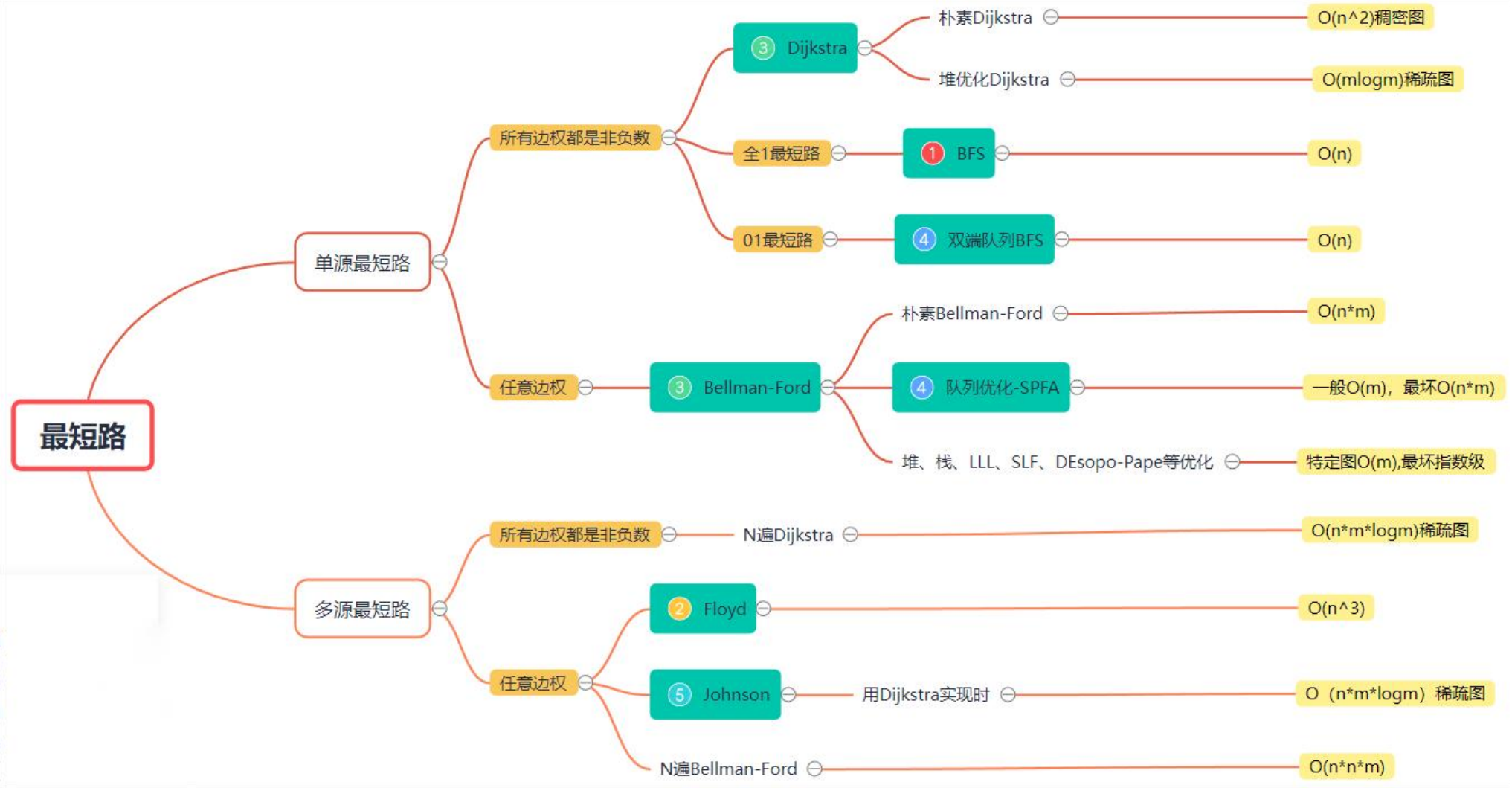
最短路问题分为两类：单源最短路和多源最短路。前者只需要求一个固定的起点到各个顶点的最短路径，后者则要求得出任意两个顶点之间的最短路径。

最短路径算法， 就是给定一个带权图，询问两个点之间代价最小的路径

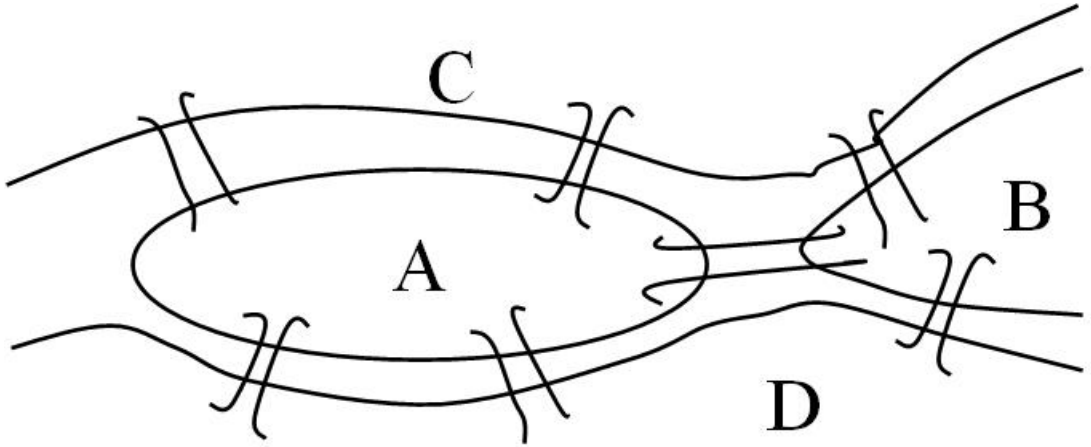
最短路

COOLMAKER 酷哥创客 人工智能教育

解决最短路问题主要有四种算法，Floyd、Dijkstra、Bellman-Ford、SPFA分别适用不同的情况。



图是由若干给定的顶点及连接两顶点的边所构成的图形，这种图形通常用来描述某些事物之间的某种特定关系。顶点用于代表事物，连接两顶点的边则用于表示两个事物间具有这种关系。图论起源于著名的**柯尼斯堡七桥问题**(下图所示)，该问题于1736年被欧拉解决，因此普遍认为**欧拉**是图论的创始人。



搜索方法：

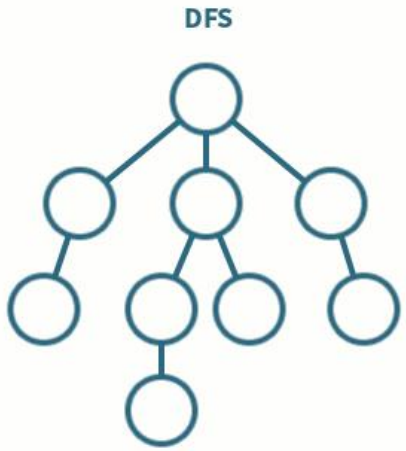
广度优先遍历

思想：

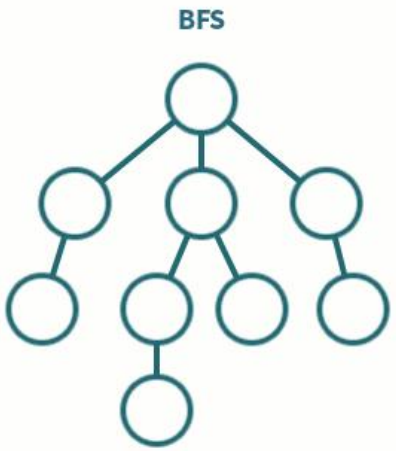
1.从起始顶点开始，先访问所有相邻顶点

2.然后再依次访问这些相邻顶点的相邻顶点

3.依此类推，直到所有顶点都被访问



深度优先搜索

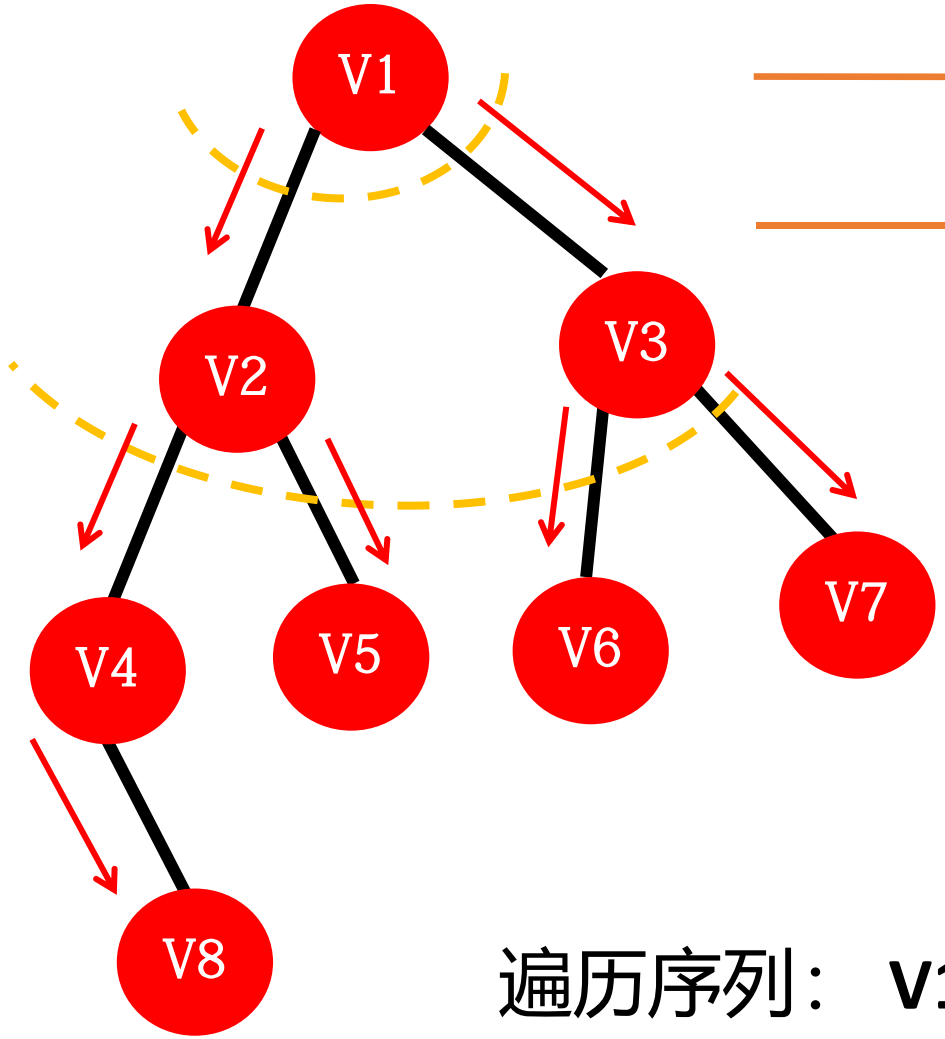


宽度优先搜索



温故知新

使用队列模拟宽度优先搜索



队列



遍历序列： V1 V2 V3 V4 V5 V6 V7 V8

题目描述

有一个很大的水缸和二个容量分别为X和Y的水壶，按照以下的规则倒水，问最少经几次倒水后，可在X或Y水壶中得到Z升水。

规则1：**水缸**向**水壶1**倒水，将水壶1装满；

规则2：**水缸**向**水壶2**倒水，将水壶2装满；

规则3：**水壶1**向**水缸**倒水，直到水壶1空；

规则4：**水壶2**向**水缸**倒水，直到水壶2空；

规则5：**水壶1**向**水壶2**倒水，直到水壶1空了或者水壶2满了；

规则6：**水壶2**向**水壶1**倒水，直到水壶2空了或者水壶1满了；

输入

只有一行数据，包括以空格分隔的三个数字，分别表示水壶1的水量(0 < 水量 <= 100)、水壶2的水量(0 < 水量 <= 100)以及期望得到的水量(0 < 水量 <=100)。

输出

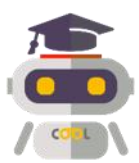
若经若干次倒水能得到所要求的水量，则输出最少的倒水次数；若无论如何倒水都无法得到规定的水量，则输出No Solution!

样例输入1 复制

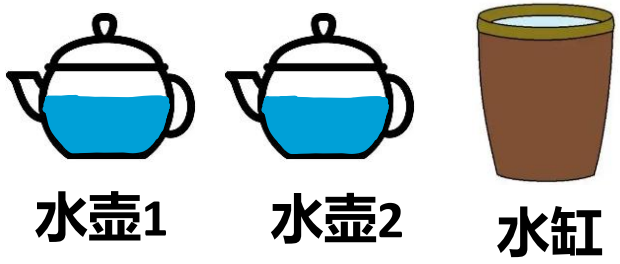
```
4 3 1
```

样例输出1 复制

```
2
```



某个时刻



规则1



规则2



规则3



规则4

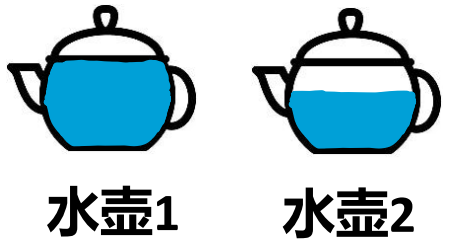


规则5



或

规则6



或

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
struct node {
    int x, y;
};
int a, b, t;
int dis[101][101];
int main() {
    cin >> a >> b >> t;
    for (int i = 0; i <= a; i++)
        for (int j = 0; j <= b; j++)
            dis[i][j] = -1;
    queue<node>q;
    q.push({0, 0});
    dis[0][0] = 0;
    while (!q.empty()) {
        node i = q.front();
        q.pop();
        if (i.x == t || i.y == t) {
            cout << dis[i.x][i.y] << "\n";
            return 0;
        }
        vector<node>nxt = {
            {a, i.y}, {i.x, b}, //装满
            {0, i.y}, {i.x, 0}, //倒空
            {max(0, i.x + i.y - b), min(b, i.x + i.y)}, //x倒向y
            {min(a, i.x + i.y), max(0, i.x + i.y - a)} //y倒向x
        };
        for (node to : nxt) {
            if (dis[to.x][to.y] != -1)continue;
            dis[to.x][to.y] = dis[i.x][i.y] + 1;
            q.push({to.x, to.y});
        }
    }
    cout << "No Solution!";
    return 0;
}
```

图的存储

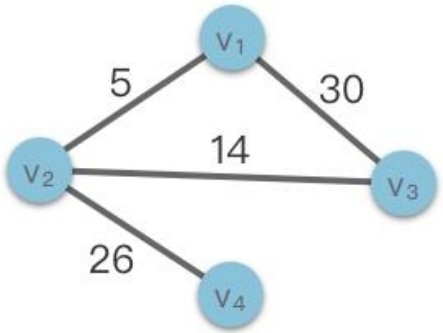


邻接矩阵

邻接矩阵的特点：

- 优点：实现简单，并且可以直接查询顶点 v_i 与 v_j 之间是否有边存在，还可以直接查询边的权值。
- 缺点：初始化效率和遍历效率较低，空间开销大，空间利用率低，并且不能存储重复边，也不便于增删节点。如果当顶点数目过大（比如当 $n > 10^5$）时，使用邻接矩阵建立一个 $n \times n$ 的二维数组不太现实。

图

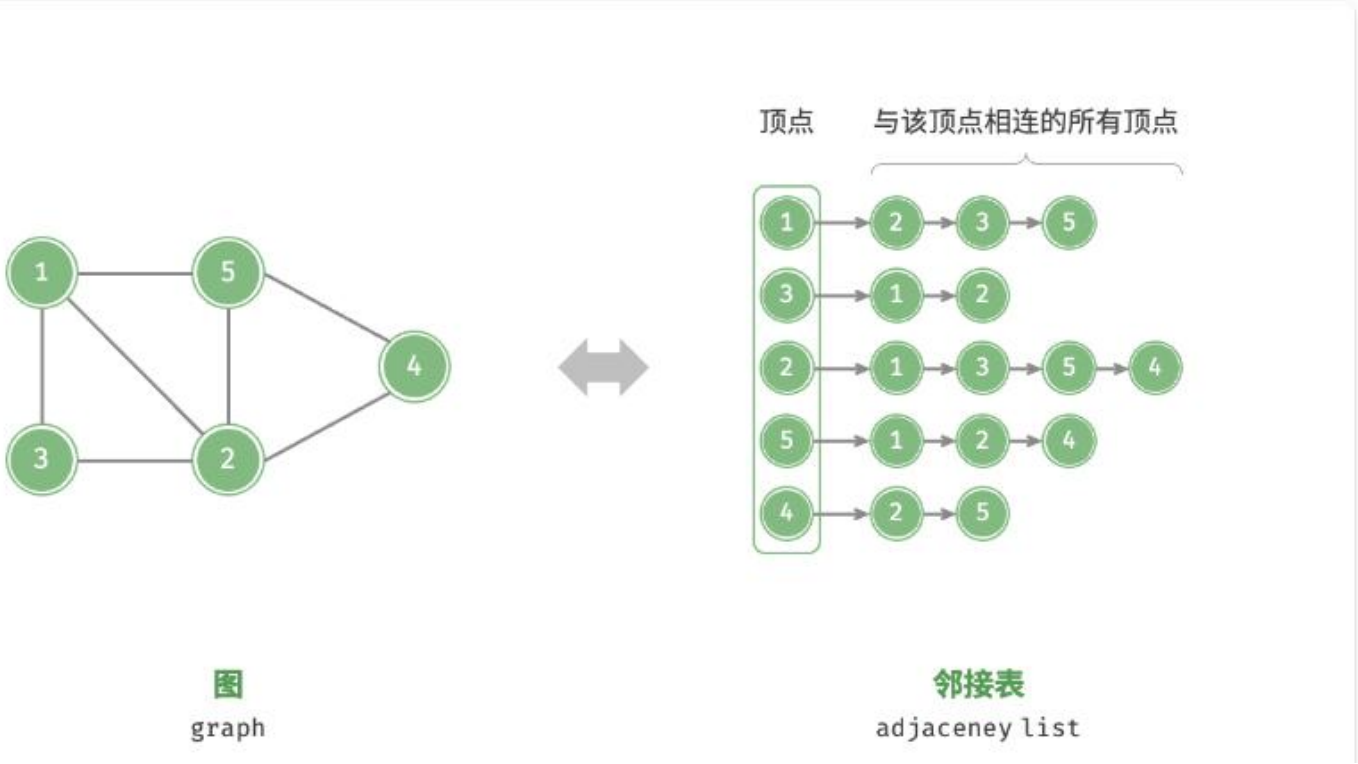


邻接矩阵

	v1	v2	v3	v4
v1	∞	5	30	∞
v2	5	∞	14	26
v3	30	14	∞	∞
v4	∞	26	∞	∞

邻接表

邻接表（adjacency list）使用 n 个链表来表示图，链表节点表示顶点。第 i 个链表对应顶点 i，其中存储了该顶点的所有邻接顶点（与该顶点相连的顶点）。图 9-6 展示了一个使用邻接表存储的图的示例。



邻接表仅存储实际存在的边，而边的总数通常远小于 n^2，因此它更加节省空间。然而，在邻接表中需要通过遍历链表来查找边，因此其时间效率不如邻接矩阵。



图论中的各类构图原理





基本建模

- 图论的学习通常是从道路/交通/行程问题开始考虑的，从“图”的角度开始思考模型
- 但是在实际应用层面，最基础也是最重要的能力就是把一般问题抽象成图，这就涉及到一些映射关系
- 图论重要组成分三部分：点+边+权值
  - 点：表示讨论的个体，可以是事件/状态/决策点
  - 边：边最重要的就是“关系”，可以理解成点和点之间的联系
  - 权值：权值通常代表具体的好处或者代价，通常和题目求解的本质直接对应



建图技巧 - 虚拟节点

- 虚拟节点
  - 当存在多个起点或多个终点时，为了能一次性运行算法（如Dijkstra, BFS, 最大流）。
  - 创造一个超级源点/汇点
- 使用场景： 多个仓库供应货物，多个商店接收货物，求最大运输量。建超级源点连接所有仓库（边权为仓库容量），超级汇点连接所有商店（边权为商店需求）



建图技巧 - 拆点

- 拆点
  - 当顶点本身有容量、代价、或者状态限制时（例如，每个点只能经过一次、经过某个点需要花费时间/金钱）。
  - 可以将一个物理节点拆分为两个或多个逻辑节点，用边来表示节点的属性。
- 使用场景：求节点有容量限制（每个点只能经过一次）的最短路径。将每个点i拆成`i_in`和`i_out`，中间连一条容量为1、边权为0的边。原图中的边`(u, v)`变为`(u_out, v_in)`，权值不变



建图技巧 - 分层图

- 分层图
  - 问题中有多个维度的状态，最典型的是“有K次机会可以忽略某种代价”（如免票、穿越障碍、逆向行驶）。
  - 建立$K+1$层完全相同的图（每层图是原问题的图）。层数代表已经使用的机会次数。
- 使用场景：一张图，有k次机会可以将某条边的权值变为0，求从1到N的最短路。建立$k+1$层图，每层内部是原图（边权为w），同时从第i层的u向第$i+1$层的v连一条权为0的边（对应原图中每条可以使用机会的边）。



建图技巧 - 边点互换

- 边点互换
  - 当问题的瓶颈在于边的数量巨大（如完全图），或者问题的核心是边与点的关系时。
  - 通常将原图的边转化为点，原图的点转化为边，或者将一组点/边合并为一个超级点。
- 使用场景：有时为了处理“必经点”或“必经边”问题，也会通过缩点或重建图的方式来简化。



建图技巧 - 反向建边

- 反向建边
  - 核心思路是改变思考的方向，从目标倒推回起点
  - 处理多源点 → 单源点
  - 处理“依赖关系”和“必要条件”：在很多问题中，原图的边表示一种依赖（如“要完成A，必须先完成B”，则边为 B -> A）。如果我们要找所有导致某个结果的原因，在正图上搜索会非常困难。而在反图上，边 A -> B 的含义变成了“如果A发生，那么B是它的一个前提条件”，这样从结果出发，就能顺藤摸瓜找到所有必要的前提。





树

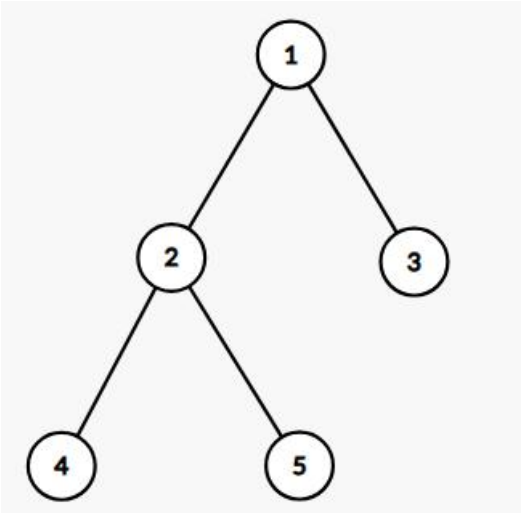


DFS序

DFS序是一个非常宽泛的概念，它泛指按照深度优先搜索顺序遍历节点所产生的序列。DFN序和欧拉序都可以看作是DFS序的一种具体形式。

但在一些特定的上下文（尤其是在国内算法竞赛社区）中，当人们单独说“DFS序”时，他们通常指的是下文将介绍的 DFN序。为了避免混淆，我们需要根据具体语境来区分。

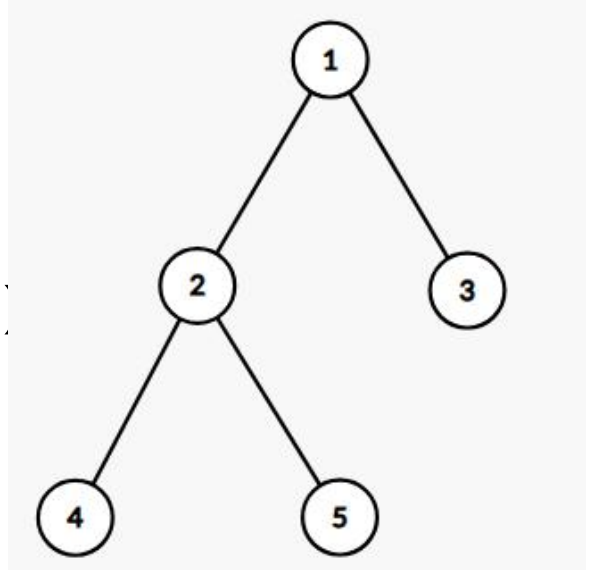
如右图的dfs序为 1 2 4 5 3 或 1 3 2 4 5



欧拉序

欧拉序在DFS遍历时，不仅在进入节点时记录，在退出节点（即回溯）时也会记录。根据记录规则的不同，主要有两种常见的欧拉序。

欧拉序1（仅第一次和最后一次）
从根节点开始DFS。第一次进入节点时记录一次。在结束递归，回溯离开该节点时再记录一次。
欧拉序：1 2 4 5 2 3 1

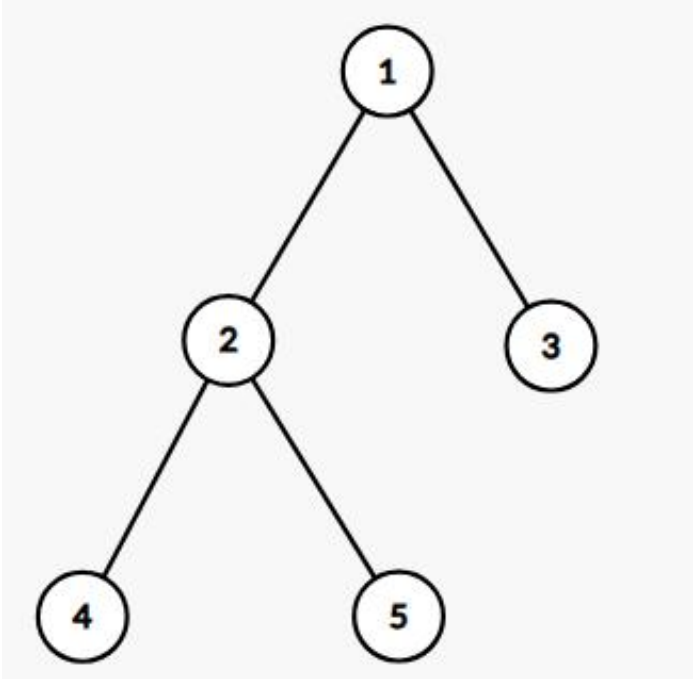


欧拉序2（所有进出都记录）可以将求LCA转为RMQ问题
从根节点开始DFS。每次访问一个节点（无论是第一次进入还是后续返回）
序列。
欧拉序：1 2 4 2 5 2 1 3 1

DFN序 (时间戳序)

COOLMAKER
酷哥创客 人工智能教育

DFN序的核心是为每个节点打上时间戳（根据DFS序/欧拉序），记录节点被访问到的顺序（时间戳）。




```
vector<int>g[1005];
int in[1005],out[1005],tot;
void dfs(int x)//得到前序的时间戳
{
    in[x]=++tot;
    for(int y:g[x])
        dfs(y);
    out[x]=tot;
}
void dfs2(int x)//得到后序的时间戳
{
    in[x]=tot+1;
    for(int y:g[x])
        dfs2(y);
    out[x]=++tot;
}
```

DFN序 (时间戳序)

COOLMAKER
酷哥创客 人工智能教育

特性与用途

子树连续：这是一条非常重要的性质。一棵子树的所有节点在DFN序中是连续存储的。如果子树根节点 u 的时间戳是 dfn[u]，子树大小为 sz[u]，那么这棵子树对应的区间就是 [dfn[u], dfn[u] + sz[u] - 1]；同样的如果第一次进入的时间戳记为 in[u]，子树中最大in[i] 记为 out[u]，那么这棵子树对应的区间就是 [in[u], out[u]]。

将树“拍平”成线性结构：利用子树连续的属性，我们可以将树上的子树问题转化为序列上的区间问题；还可以结合线段树或ST表等数据结构来进行高效的查询和修改。



典型应用：子树所有节点的权值之和、最大值、最小值；给子树所有节点增加一个值。



倍增





倍增思想

COOLMAKER
酷哥创客 人工智能教育

倍增（binary lifting），顾名思义，就是成倍的增加，“以2幂的成倍的增长”的思想来解决问题的一类算法。，能够使得**线性的处理**转化为**对数级的处理**，大大地优化时间复杂度；在很多算法中都有应用，其中最常见的就是快速幂、ST表、树上最近公共祖先LCA、后缀数组SA

假设我们用 f 来表示我们想要求解的问题，用 $f(x)$来表示【规模为x的问题f的解】。
如果 f 具有某些性质，使得我们可以在已经求得了 $f(x)$的情况下快速的求得 $f(2x)$，并且我们能够比较快速求得f(1)，那么我们就可以通过递推的方式依次快速的求得 $f(2)$、$f(4)$、……等等形如 $f(2b)$的值，并且如果f能够具有**可加性**的话（即x=x1+x2 且 $f(x)=f(x1)\oplus f(x2)$），倍增法就可以发挥巨大的威力

如果 f 具有可加性的话，那么我们就可以得到以下推导：

1. 利用倍增法，我们可以快速得到所有形如 $f(2^b)$ 的值。
2. 对于任意问题规模 x，我们可以将 x 拆分为两两不同的2的非负整数次幂之和，即 $x = 2^{b_1} + 2^{b_2} + \ldots + 2^{b_m}$，其中 $0 \le b_1 < b_2 < \ldots < b_m$。
3. 利用 f 的可加性，可知 $f(x) = f(2^{b_1}) \bigoplus f(2^{b_2}) \bigoplus \ldots \bigoplus f(2^{b_m})$，即把 $f(x)$ 拆分成了若干个 $f(2^b)$ 的广义和。



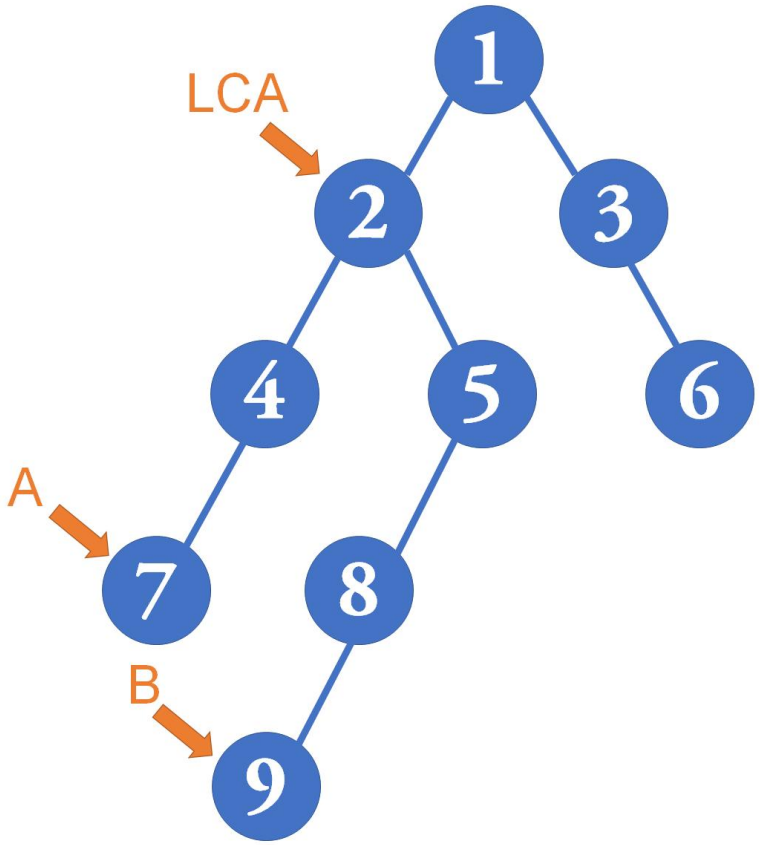
最近公共祖先

最近公共祖先（Lowest Common Ancestors，LCA）是指在一棵有根树中，距离两个节点最近的公共祖先节点。

最近公共祖先表示的是x和y的所有**公共祖先中深度最大的一个**；它也可以被理解为**x到根的路径与y到根的路径的交汇点**；或者说是**x与y之间的路径上深度最小的节点。**

求解最近公共祖先的方法有

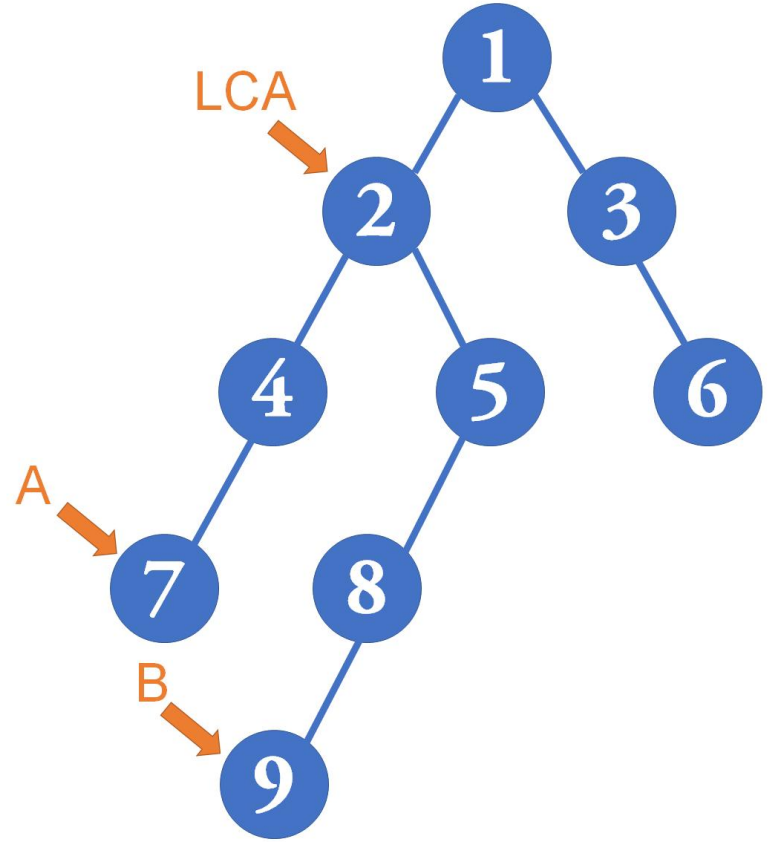
- 朴素解法
- 倍增算法
- Tarjan 算法
- 树链剖分
- 用DFS序转换为RMQ问题
- 用欧拉序列转换为RMQ问题
- 动态树



最近公共祖先的性质

我们记某点集 S={v1,v2,...,vn}的最近公共祖先为 LCA({v1,v2,...,vn}) 或 LCA(S)。

1. $\mathrm{LCA}(\{u\}) = u$;
2. u 是 v 的祖先，当且仅当 $\mathrm{LCA}(u, v) = u$;
3. 如果 u 不为 v 的祖先并且 v 不为 u 的祖先，那么 u, v 分别处于 $\mathrm{LCA}(u, v)$ 的两棵不同子树中;
4. 前序遍历中，$\mathrm{LCA}(S)$ 出现在所有 S 中元素之前，后序遍历中 $\mathrm{LCA}(S)$ 则出现在所有 S 中元素之后;
5. 两点集并的最近公共祖先为两点集分别的最近公共祖先的最近公共祖先，即 $\mathrm{LCA}(A \cup B) = \mathrm{LCA}(\mathrm{LCA}(A), \mathrm{LCA}(B))$; 是可重复贡献问题
6. 两点的最近公共祖先必定处在树上两点间的最短路上;
7. $d(u, v) = h(u) + h(v) - 2h(\mathrm{LCA}(u, v))$，其中 d 是树上两点间的距离，h 代表某点到树根的距离。 两点间路径距离公式



倍增求最近公共祖先

我们先来讨论朴素的做法

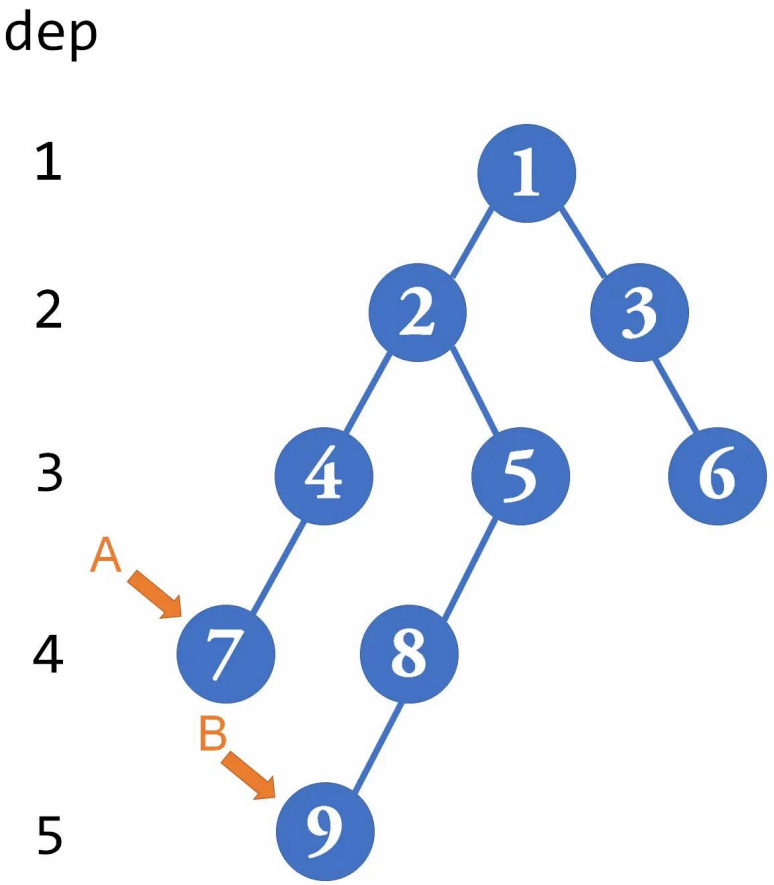
1、首先DFS遍历整棵树，求出每个点的深度

2、现在A点的深度比B点小，所以我们先让B点往上“爬”，爬到与A点深度相等为止。

3、然后A点和B点再一起往上爬，直到两点相遇，那一点即为LCA

这样下来，每次查询LCA的最坏时间复杂度是O(n)的。

有时候，我们需要进行很多次查询，这时朴素算法时间复杂度就不够用了。我们考虑空间换时间的倍增算法。



倍增求最近公共祖先

预处理阶段：

倍增求LCA与求解ST表类似。我们用一个数组dp[i][k]存储号i点的2^k级祖先。（父节点为1级祖先，祖父结点为2级祖先……以此类推）

这样，往上爬的次数可以被大大缩短（现在变成“跳”了）。

```
void dfs(int x,int fa)
{
    deep[x]=deep[fa]+1;
    dp[x][0]=fa;
    for(int i=1; (1<<i)<=deep[x]; i++)
    {
        dp[x][i]=dp[dp[x][i-1]][i-1];
    }
    for(int y:v[x])
    {
        if(y==fa)continue;
        dfs(y,x);
    }
}
```

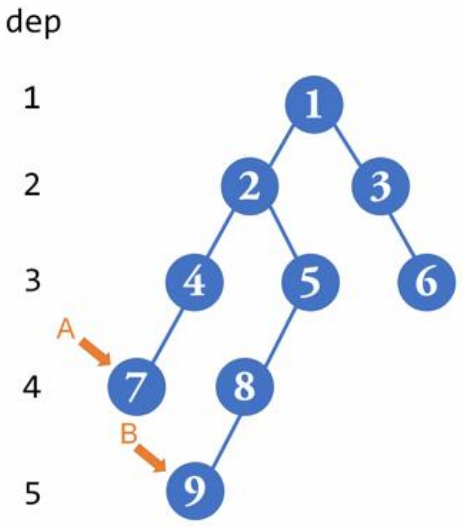


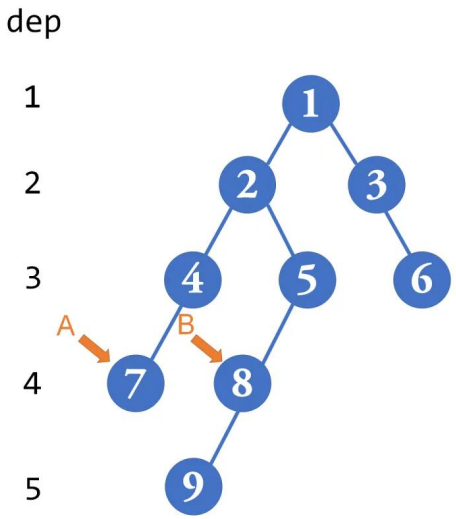
倍增求最近公共祖先

求LCA：

1、倍增首先还是先让两点深度相等，和朴素暴力的方法一样，但是现在是“跳”了
例如，a和b本来相差22的深度，现在b不用往上爬22次，只需要依次跳16、4、2个单位，3次便能达到与a相同的距离。

```
for(int i=20; i>=0; i--)//让b走到和a相同高度
{
    if(deep[a]<=deep[b]-(1<<i))
        b=dp[b][i];
}
```





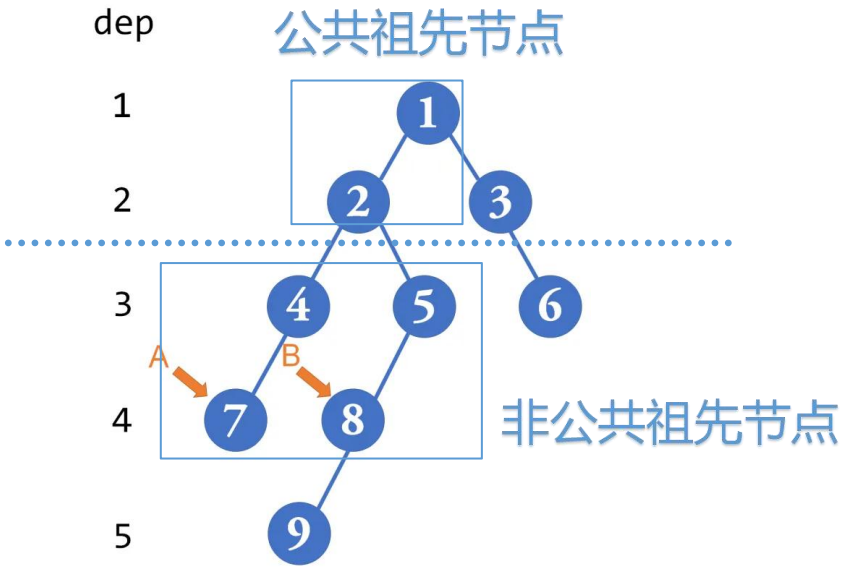


COOLMAKER 酷哥创客 人工智能教育

倍增求最近公共祖先

2、两者深度相等后，如果两个点已经相遇，那么问题就得以解决。
如果尚未相遇，我们再让它们一起往上跳。问题在于，如何确定每次要跳多少？正面解决也许不太容易，我们逆向思考：如何在a、b不相遇的情况下跳到尽可能高的位置？如果找到了这个位置，它的父亲就是LCA了。
接下来就和“找数”这题类似了，利用类似二分的思路找到一个分割点，将序列分为两个类别即可，一类是非公共祖先节点，一类是公共祖先节点，从可能跳的最大步数开始，不断减半步数往上跳找到为深度最浅的非公共祖先的位置。

```
if(a==b)return a;//如果已经找到，那么现在a就是lca了
for(int i=20; i>=0; i--)//肩并肩往上走，找到最近小于LCA的位置
{
    if(dp[a][i]!=dp[b][i])
    {
        a=dp[a][i];
        b=dp[b][i];
    }
}
return dp[a][0];//此时往上走一步就是LCA
```



```cpp
struct beizeng
{
    int dp[maxn][22],deep[maxn];
    vector<int>v[maxn];
    void add(int a,int b)
    {
        v[a].push_back(b);
        v[b].push_back(a);
    }
    void dfs(int x,int fa)
    {
        deep[x]=deep[fa]+1;
        dp[x][0]=fa;
        for(int i=1; (1<<i)<=deep[x]; i++)
        {
            dp[x][i]=dp[dp[x][i-1]][i-1];
        }
        for(int y:v[x])
        {
            if(y==fa)continue;
            dfs(y,x);
        }
    }
    int lca(int a,int b)
    {
        if(deep[a]>deep[b])swap(a,b);//默认让b深度更深
        for(int i=20; i>=0; i--)//让b走到和a相同高度
        {
            if(deep[a]<=deep[b]-(1<<i))
                b=dp[b][i];
        }
        if(a==b)return a;//如果已经找到，那么现在a就是lca了
        for(int i=20; i>=0; i--)//肩并肩往上走，找到最近小于LCA的位置
        {
            if(dp[a][i]!=dp[b][i])//不是LCA就往上走
            {
                a=dp[a][i];
                b=dp[b][i];
            }
        }
        return dp[a][0];//此时往上走一步就是LCA
    }
} tree;
```

故输出依次为 4,4,1,4,4。

COOLMAKER

```cpp
#include <bits/stdc++.h>
using namespace std;
const int maxn=5e5+10;
struct beizeng
{
    int dp[maxn][22],deep[maxn];
    vector<int>v[maxn];
    void add(int a,int b)
    {...}
    void dfs(int x,int fa)
    {...}
    int lca(int a,int b)
    {...}
} tree;
int n,m,s;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>m>>s;
    for(int i=2,x,y; i<=n; i++) //n-1条边
    {
        cin>>x>>y;
        tree.add(x,y);
    }
    tree.dfs(s,-1);
    for(int i=1,x,y; i<=m; i++)
    {
        cin>>x>>y;
        cout<<tree.lca(x,y)<<"\n";
    }
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

```
4   struct beizeng
5   {
6       int dp[maxn][22],deep[maxn];
7       vector<int>v[maxn];
8       void add(int a,int b)
9       {
10          v[a].push_back(b);
11          v[b].push_back(a);
12      }
13      void dfs(int x,int fa)
14      {
15          deep[x]=deep[fa]+1;
16          dp[x][0]=fa;
17          for(int i=1; (1<<i)<=deep[x]; i++)
18          {
19              dp[x][i]=dp[dp[x][i-1]][i-1];
20          }
21          for(int y:v[x])
22          {
23              if(y==fa)continue;
24              dfs(y,x);
25          }
26      }
```

```
27      int lca(int a,int b)
28      {
29          if(deep[a]>deep[b])swap(a,b);//默认让b深度更深
30          for(int i=20; i>=0; i--)//让b走到和a相同高度
31          {
32              if(deep[a]<=deep[b]-(1<<i))
33                  b=dp[b][i];
34          }
35          if(a==b)return a;//如果已经找到，那么现在a就是lca了
36          for(int i=20; i>=0; i--)//肩并肩往上走，找到最近小于LCA的位置
37          {
38              if(dp[a][i]!=dp[b][i])//不是LCA就往上走
39              {
40                  a=dp[a][i];
41                  b=dp[b][i];
42              }
43          }
44          return dp[a][0];//此时往上走一步就是LCA
45      }
46  } tree;
```



并查集基础





什么是并查集

正如它的名字一样，**并查集**就是用来对集合进行 **合并** 与 **查询** 操作的一种数据结构。

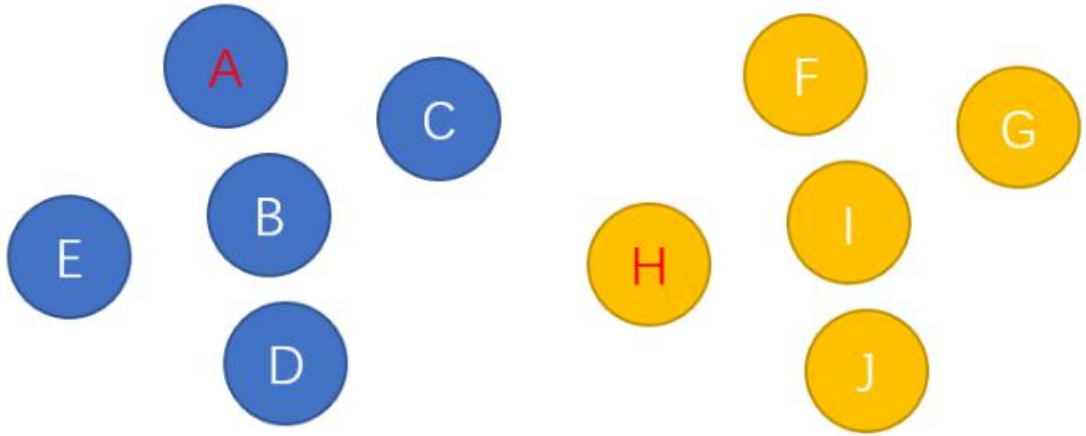
合并： 就是将两个不相交的集合合并成一个集合。

查询： 就是查询两个元素是否属于同一集合。



并查集的优越性

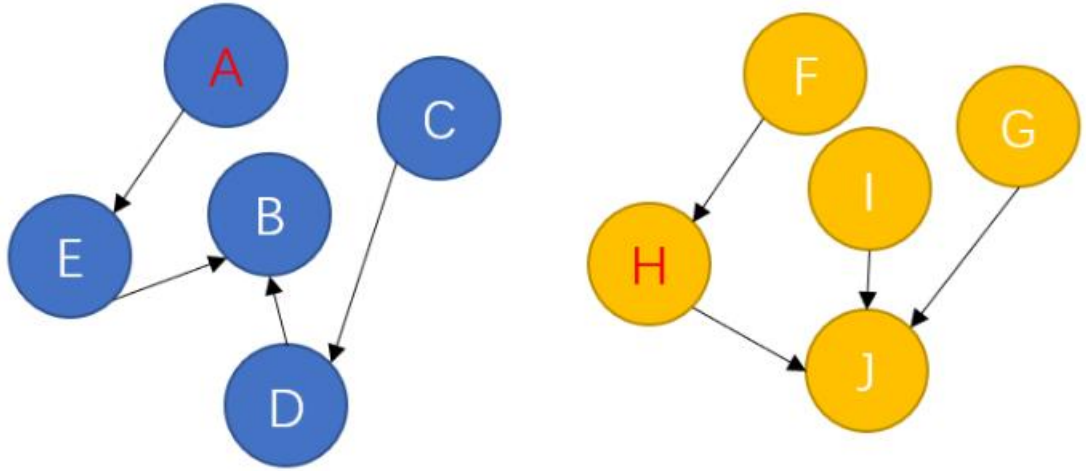
对于如下图所示的两个集合，如果我们要判断H和A是否在同一个集合中，我们需要遍历A所在的集合，并逐一判断当前节点是否是H节点，直到最后遍历完整个蓝色集合，才能判断出H节点不在这个集合中。



同样的，如果我们需要合并两个集合，就需要遍历整个黄色的集合，将里面的节点一个一个加入到蓝色集合中。两者都是O(N)的复杂度。

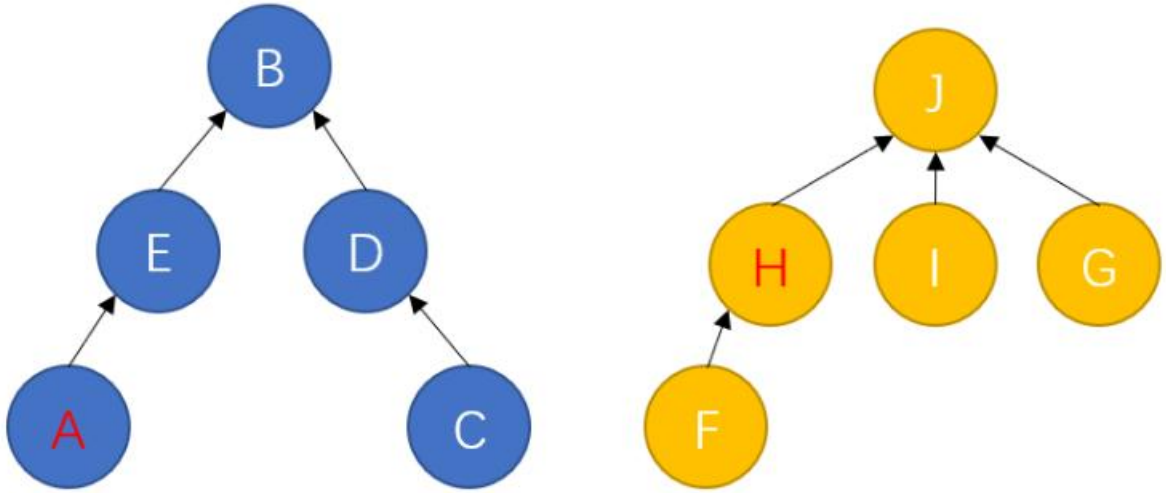
并查集的优越性

但倘若我们在生成集合的时候，就人为地将集合中的元素之间创建某种关联，使它们具有共同的头结点，那么查询和合并的操作将会省时很多。
就拿刚刚的两个集合举例，在创建集合的过程中，为节点之间创建“联系”，形成如下图的结构：



并查集的优越性

可以发现，最终生成的这个结构其实就是一个树形结构。



这也就意味着一个集合中的所有节点都可以找到同一个头结点。此时合并和查询操作将变得异常简单：

查询：只需要判断两个元素是否具有相同的头结点。

合并：只需要将一个集合的头结点挂到另一个集合的头结点下即可。

可以发现，上述两个操作的时间复杂度都与“获取头结点”这一过程，也就是树的高度有关。因此，假如生成的树只有有限高度的话，合并和查询的操作都是 O (1)的时间复杂度。

COOLMAKER 酷哥创客 人工智能教育

下面我们来讲讲如何代码实现并查集

1. 定义数组pa[]，代表每个节点的头节点是谁。

```
int pa[10005];
```

2. 初始化每个节点的头节点是其本身。

```
for(int i=1;i<=n;i++)pa[i]=i;
```

3. 编写find_set函数，用于查询当前节点的头节点

```
int find_set(int x)
{
    if(x==pa[x])return x;
    return find_set(pa[x]);
}
```

4. 编写merge_set函数，用于合并2个节点的集合

```
void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        pa[fa]=fb;
    }
}
```



2599: 【模板】并查集

时间限制: 1 Sec 内存限制: 256 MB

提交: 0 解决: 5

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

题目描述

有n个元素，你需要完成集合的合并和判断两个元素是否在同一集合中这两种操作。

输入

第一行包含两个整数n,m，表示共有n个元素和m次操作。**(1<=n<=1e4,1<=m<=1e5)**

接下来m行，每行包含三个整数z_i,x_i,y_i。**(1<=z<=2,1<=x,y<=n)**

当z_i=1时，将x_i与y_i所在的集合合并。当z_i=2时，输出x_i与y_i是否在同一集合内，是的话输出 Y；否则输出 N。

本题输入输出过多

c++请用scanf输入printf输出,或者加入 ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);这段语句

输出

对于每一个z_i=2的操作，都有一行输出，每行包含一个大写字母，为 Y 或者 N。

样例输入

```
4 7
2 1 2
1 1 2
2 1 2
1 3 4
2 1 4
1 2 3
2 1 4
```

样例输出

```
N
Y
N
Y
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,m;
int pa[10005];

int find_set(int x)
{
    if(x==pa[x])return x;
    return find_set(pa[x]);
}

void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        pa[fa]=fb;
    }
}

int main()
{
    scanf("%d %d",&n,&m);

    for(int i=1;i<=n;i++)pa[i]=i;

    for(int i=1,op,x,y;i<=m;i++)
    {
        scanf("%d%d%d",&op,&x,&y);
        if(op==1)
        {
            merge_set(x,y);
        }else
        {
            if(find_set(x)==find_set(y))
            {
                printf("Y\n");
            }else
            {
                printf("N\n");
            }
        }
    }
    return 0;
}
```

我们提交后会发现是：

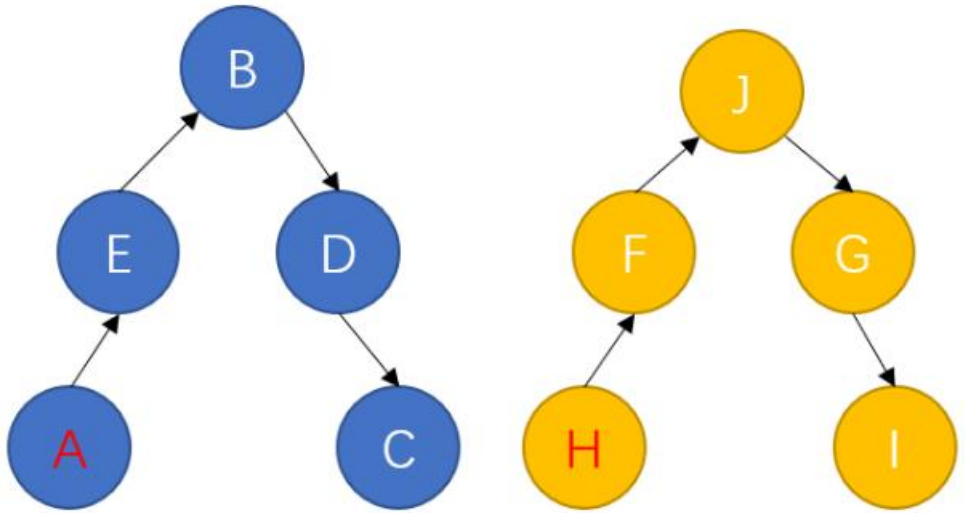


这是为什么？



并查集的优化

假如生成的树的高度与集合元素个数相同，那合并和查询操作的时间复杂度就和直接遍历的方式差不多。



在上图所示的最差情况下，合并和查询的时间复杂度都是 O (N) 。
因此，如何减少树的高度直接决定着并查集的性能如何。
那么如何尽量减少树的高度呢?

并查集的优化

按秩合并和路径压缩并查集是两个很常见的优化方法，用于加速并查集的操作。

按秩合并指的是将较小的树合并到较大的树上，从而减少树的深度，降低后续查找和合并的时间复杂度；同理，也可以将较矮的树合并到较高的树上。

路径压缩则是在查找根节点的过程中对路径进行压缩，使得查找根节点的速度更快，也是最常用的并查集优化方法。

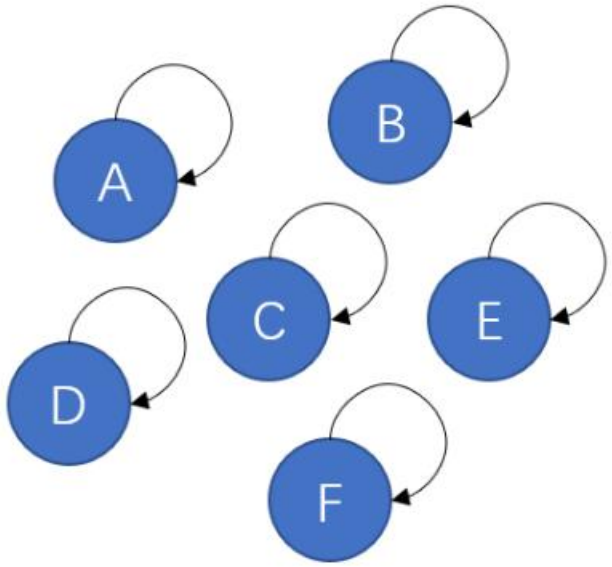
这两个优化方法可以一起使用，其中按秩合并可以提高合并操作的效率，路径压缩可以提高查找操作的效率。使用这两个优化方法后，可以将并查集的单次查找、合并操作的时间复杂度均优化到O(log n)左右。



并查集的优化

按秩合并

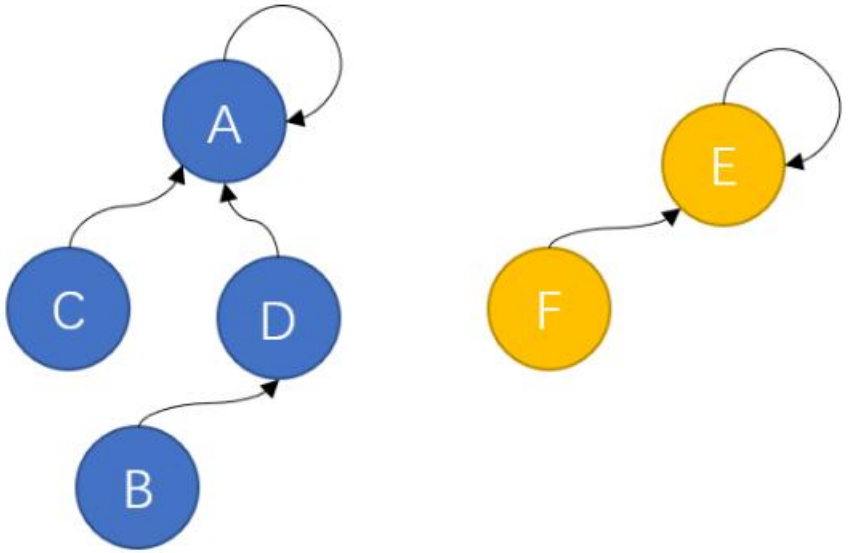
在最原始的状态下，每个点自己就是一个集合，它们的指针都指向自己。就像是一片草原上的几个原始部落，一开始他们之间毫无瓜葛，各自为政，自己就是自己的主人。



并查集的优化

按秩合并

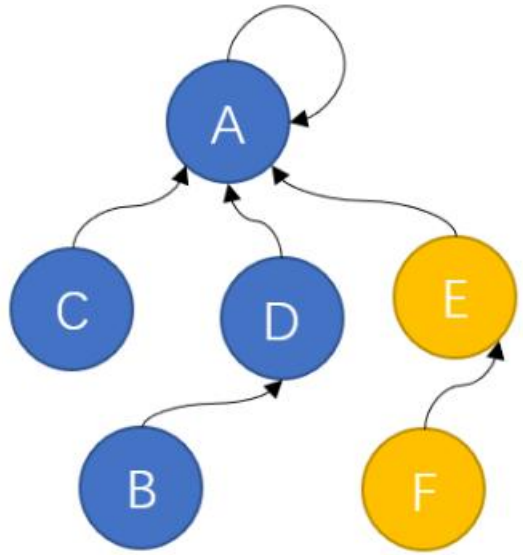
但这片草原一共就这么大点，随着某几个部落逐渐繁荣兴盛，他们的地盘也愈发显得局促，因此他们开始互相征战，吞并其他的部落形成更大的部落。在战争中落败的一方就认另一方作为首领。



并查集的优化

按秩合并

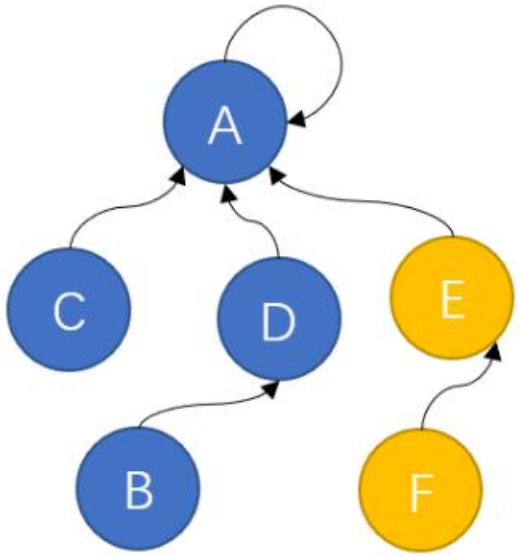
经过几轮兼并，现在草原上只剩下两个规模比较大的部落，此时两个部落继续进行战争，蓝色方凭借自身庞大的兵力轻松取胜，将黄色方纳入麾下，并形成如下的组织结构：



并查集的优化

按秩合并

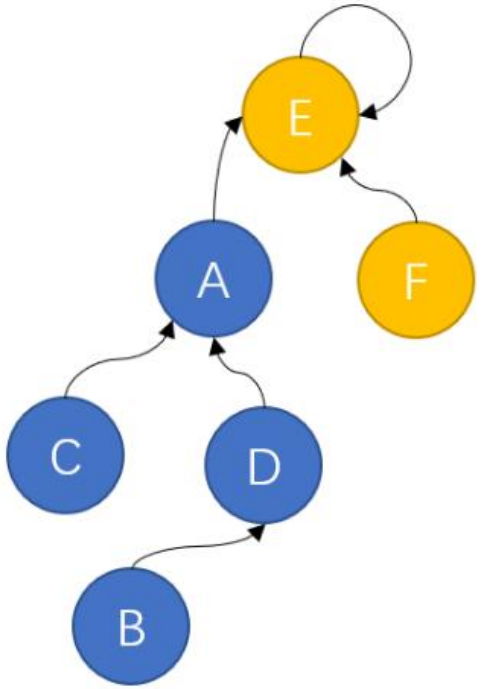
组织上离最高首领最远的B、F两个部落如果想要向A传达信息，只需要经过自己的上司D或E一个节点就可以。



并查集的优化

按秩合并

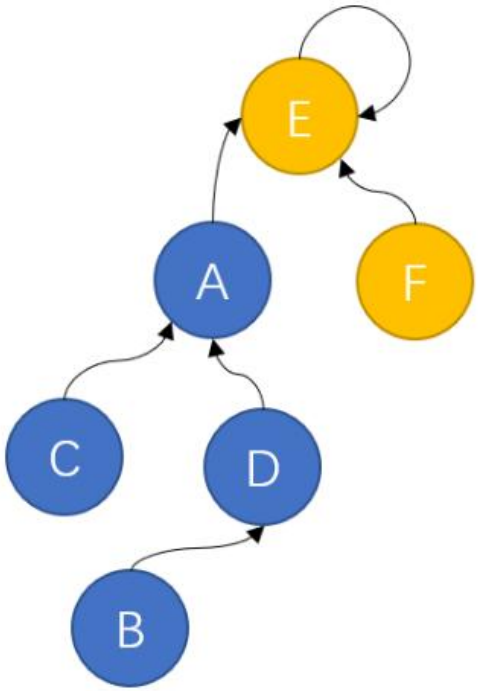
但战争往往不总是尽如人意，黄色方偏偏就凭借战士们顽强的意志，拿下了这场战争的胜利，将蓝色方吞并。此时就会形成如下的组织结构：



并查集的优化

按秩合并

这时就出现问题了，距离最高首领最远的B如果想要向E传达消息，就需要经过D、 A两个上司，这样的效率显然不如之前一种组织结构。



并查集的优化

按秩合并

我们的并查集也是如此，当两棵深度不同的树进行合并时，往往将深度较小的树挂载到深度较大的树下，因为这样形成的树深度更小，在寻找头结点时也就有更高的效率。

在执行合并操作时，将更小的树连接到更大的树上，这样的优化方式就称为“按秩合并”



并查集的优化

按秩合并

按秩合并是在 merge_set操作中实现的。
主要做法是：使具有较少节点的树的根指向具有较多节点的树的根。
这里节点的数量可以通过维护一个秩来表示的，秩表示该节点高度的一个上界。
在使用按秩合并的 merge_set操作中，我们可以让具有较小秩的树的根指向具有较大秩的树的根。按秩合并的优化主要是对查询的优化，可以减少查询次数。



并查集的优化

按秩合并

1 因为让具有较小秩的树的根指向具有较大秩的树的根，所以我们要先定义一个数组sz[],表示目前这个树的大小

```
int pa[10005];
```



```
int pa[10005],sz[10005];
```

2 同理，我们还需初始化，sz[]数组的值

```
for(int i=1;i<=n;i++)pa[i]=i;
```



```
for(int i=1;i<=n;i++)pa[i]=i,sz[i]=1;
```



并查集的优化

按秩合并

3 在merge_set函数里较小的树的根指向较大的树的根

```
void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        pa[fa]=fb;
    }
}
```



```
void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        if(sz[fa]>sz[fb]) sz[fa]+=sz[fb],pa[fb]=fa;
        else sz[fb]+=sz[fa],pa[fa]=fb;
    }
}
```



2599: 【模板】并查集

时间限制: 1 Sec 内存限制: 256 MB

提交: 0 解决: 5

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

题目描述

有n个元素，你需要完成集合的合并和判断两个元素是否在同一集合中这两种操作。

输入

第一行包含两个整数n,m，表示共有n个元素和m次操作。**(1<=n<=1e4,1<=m<=1e5)**

接下来m行，每行包含三个整数z_i,x_i,y_i。**(1<=z<=2,1<=x,y<=n)**

当z_i=1时，将x_i与y_i所在的集合合并。当z_i=2时，输出x_i与y_i是否在同一集合内，是的话输出 Y；否则输出 N。

本题输入输出过多

c++请用scanf输入printf输出,或者加入 ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);这段语句

输出

对于每一个z_i=2的操作，都有一行输出，每行包含一个大写字母，为 Y 或者 N。

样例输入

```
4 7
2 1 2
1 1 2
2 1 2
1 3 4
2 1 4
1 2 3
2 1 4
```

样例输出

```
N
Y
N
Y
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,m;
int pa[10005],sz[10005];

int find_set(int x)
{
    if(x==pa[x])return x;
    return find_set(pa[x]);
}

void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        if(sz[fa]>sz[fb]) sz[fa]+=sz[fb],pa[fb]=fa;
        else sz[fb]+=sz[fa],pa[fa]=fb;
    }
}

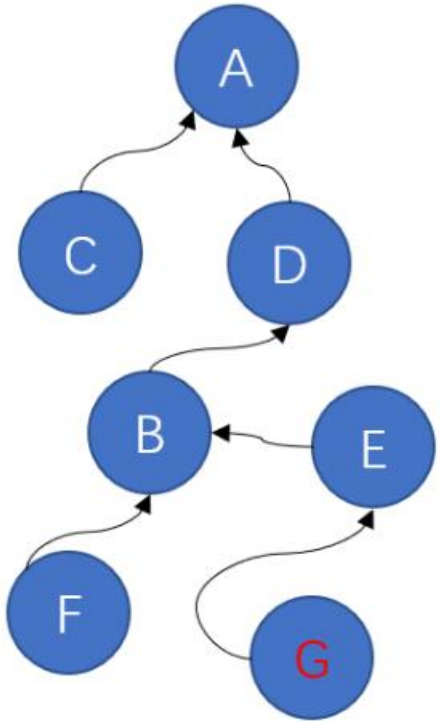
int main()
{
    scanf("%d %d",&n,&m);

    for(int i=1;i<=n;i++)pa[i]=i,sz[i]=1;

    for(int i=1,op,x,y;i<=m;i++)
    {
        scanf("%d%d%d",&op,&x,&y);
        if(op==1)
        {
            merge_set(x,y);
        }else
        {
            if(find_set(x)==find_set(y))
            {
                printf("Y\n");
            }else
            {
                printf("N\n");
            }
        }
    }
    return 0;
}
```

并查集的优化

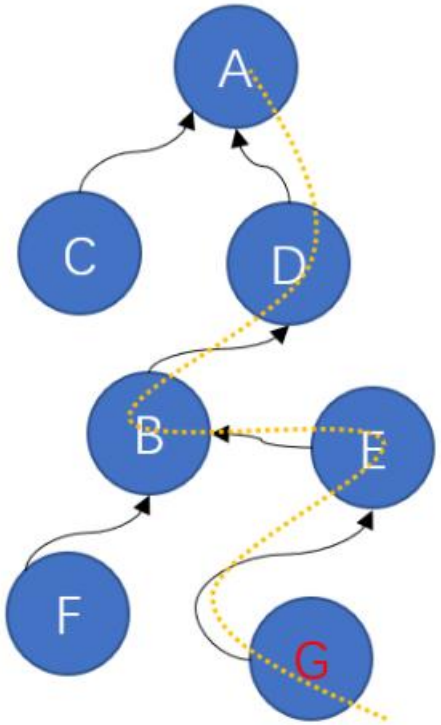
路径压缩



随着部落日渐壮大，组织结构也越来越复杂，最底层的部落如果要向最高首领传递信息，需要经过好几个中间部落。此时我们的最高首领觉得自己的统治地位受到了威胁，因为中间经过的节点越多，自己对底层部落的控制力就越弱。所以他要想办法将底层部落的控制权全都收归自己所有。

并查集的优化

路径压缩

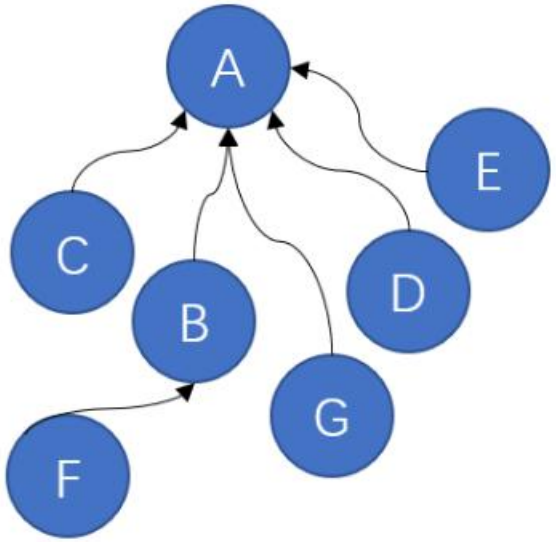


但首领并不知道自己麾下到底有多少部落，所以他颁布了一条法令：有任何部落要跟他汇报信息，都要带上他的上司一起来，他的上司也要带上他上司的上司 ……并且以后他们都直接向最高首领进行汇报，不用再经过其他节点。

此时上图中的部落G想要向A汇报信息，沿途会经过E、B、D，最后到达A

并查集的优化

路径压缩



按照规定，从今往后，G、E、B、D都直接向A汇报，无需再经过其他节点。

此时，树的高度就减小了很多，效率也会大大提升。

在执行查找的过程中，扁平化树的结构，这样的优化方式称为“路径压缩”

并查集的优化

路径压缩

路径压缩是在 find_set 操作中实现的。
主要做法是：将所有节点指向根。对于一个集合树来说，它的根节点下面可以依附着许多的节点，因此，我们可以尝试在 find_set 的过程中，从底向上，如果此时访问的节点不是根节点的话，那么我们可以把这个节点尽量的往上挪一挪，挪到根节点下面，减少树的层数。

```
int find_set(int x)
{
    if(x==pa[x])return x;
    return find_set(pa[x]);
}
```



```
int find_set(int x)
{
    if(x==pa[x])return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
```



2599: 【模板】并查集

时间限制: 1 Sec 内存限制: 256 MB

提交: 0 解决: 5

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

题目描述

有n个元素，你需要完成集合的合并和判断两个元素是否在同一集合中这两种操作。

输入

第一行包含两个整数n,m，表示共有n个元素和m次操作。**(1<=n<=1e4,1<=m<=1e5)**

接下来m行，每行包含三个整数z_i,x_i,y_i。**(1<=z<=2,1<=x,y<=n)**

当z_i=1时，将x_i与y_i所在的集合合并。当z_i=2时，输出x_i与y_i是否在同一集合内，是的话输出 Y；否则输出 N。

本题输入输出过多

c++请用scanf输入printf输出,或者加入 ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);这段语句

输出

对于每一个z_i=2的操作，都有一行输出，每行包含一个大写字母，为 Y 或者 N。

样例输入

```
4 7
2 1 2
1 1 2
2 1 2
1 3 4
2 1 4
1 2 3
2 1 4
```

样例输出

```
N
Y
N
Y
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,m;
int pa[10005];

int find_set(int x)
{
    if(x==pa[x])return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}

void merge_set(int a,int b)
{
    int fa=find_set(a);
    int fb=find_set(b);
    if(fa!=fb)
    {
        pa[fa]=fb;
    }
}

int main()
{
    scanf("%d %d",&n,&m);

    for(int i=1;i<=n;i++)pa[i]=i;

    for(int i=1,op,x,y;i<=m;i++)
    {
        scanf("%d%d%d",&op,&x,&y);
        if(op==1)
        {
            merge_set(x,y);
        }else
        {
            if(find_set(x)==find_set(y))
            {
                printf("Y\n");
            }else
            {
                printf("N\n");
            }
        }
    }
    return 0;
}
```



最小生成树Kruskal算法

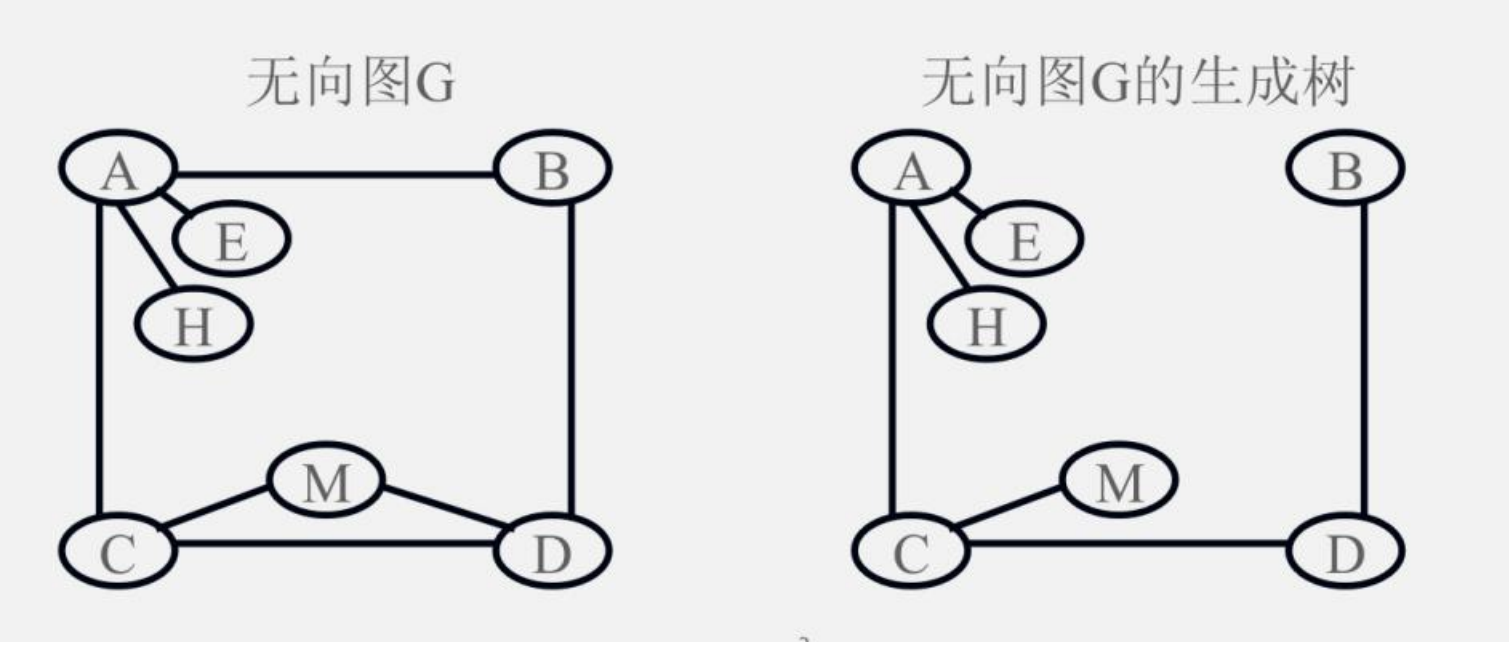




什么是生成树

定义:

生成树是无向连通图的极小连通子图。n 个结点的图，但只有n-1 条边，也就是形成了一棵树。在生成树中添加一条边之后，必定会形成回路或环。

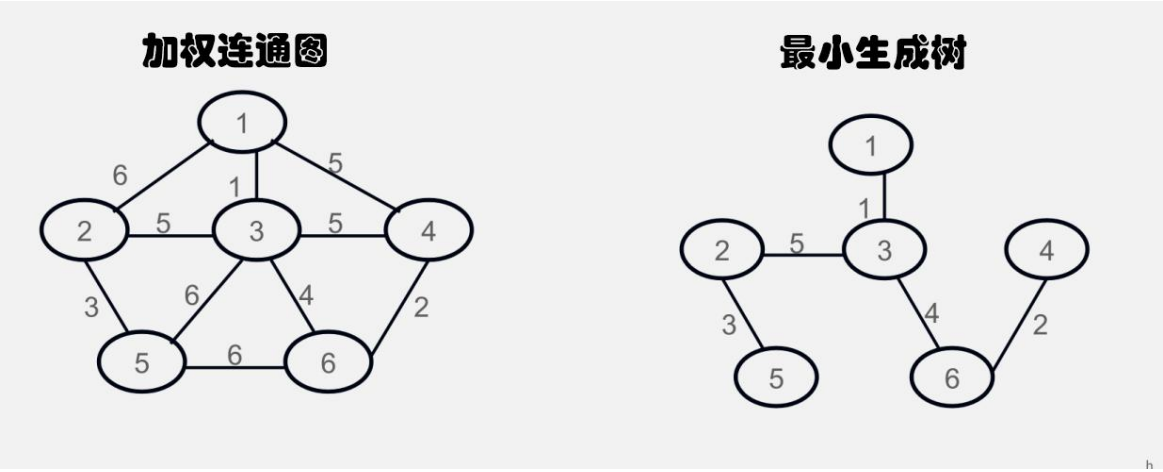


什么是最小生成树

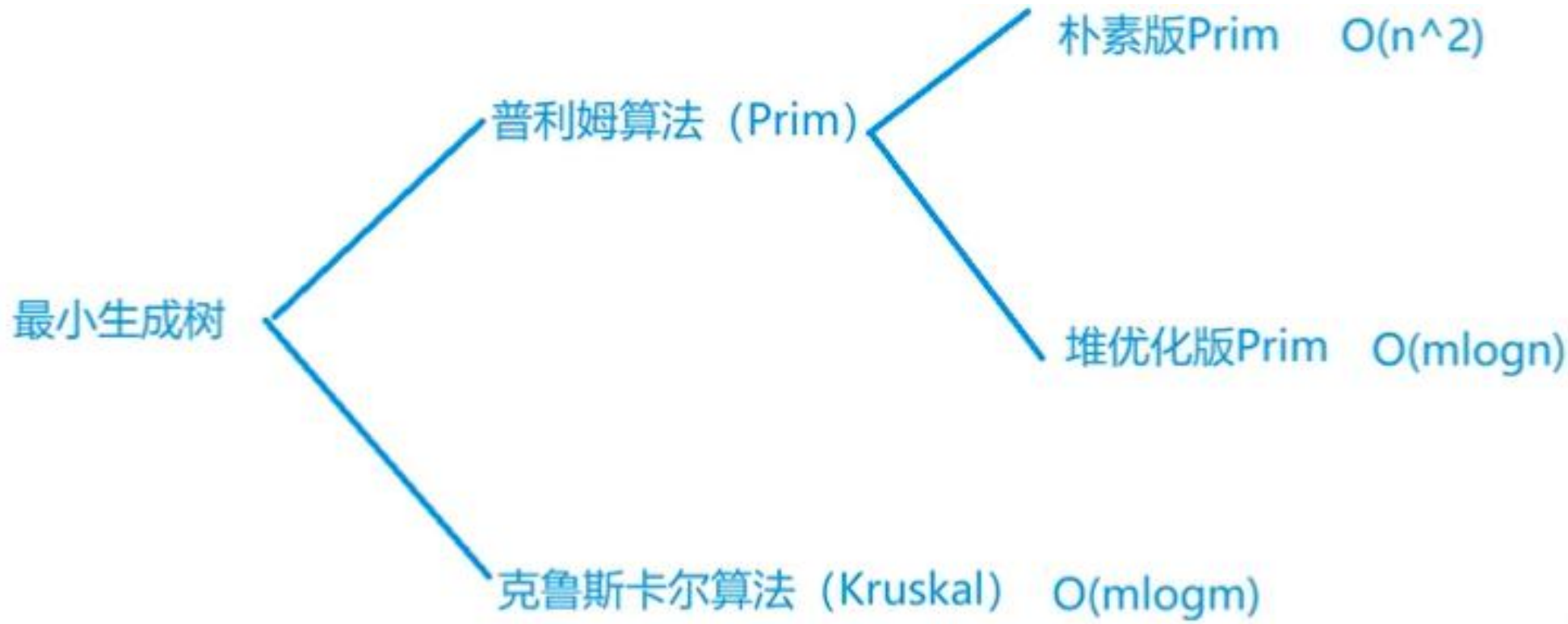
定义:

加权无向图的所有生成树中边的权值之和最小的树

最小生成树（Minimum Spanning Tree，简称MST）是一种在加权连通图中求解连通子图问题算法。该子图的所有边的权重和最小，并且每个节点都被包含在内。
最小生成树可以用于多种场景，比如：要在n个城市之间铺设光缆，主要目标是要使这 n 个城市的任意两个之间都可以通信，但铺设光缆的费用很高，且各个城市之间铺设光缆的费用不同，因此另一个目标是要使铺设光缆的总费用最低，这就是就是最小生成树问题。



实现最小生成树的两种算法



Kruskal算法的原理

下面我们来模拟一下这个过程吧！！！

1. 将一个连通块当做一个集合，Kruskal首先将所有的边按**从小到大顺序排序**（一般使用sort或者优先队列），并认为每一个点都是孤立的，分属于n个独立的集合。

2. 然后按顺序枚举每一条边。如果这条边连接着两个点属于不同的集合（并查集），那么就把这条边加入最小生成树，这两个不同的集合就合并成了一个集合。如果这条边连接的两个点属于同一集合，就跳过，

3. 直到选取了n-1条边为止，这样，边依次加入的过程中，就能够逐渐扩展出一个最小生成树。

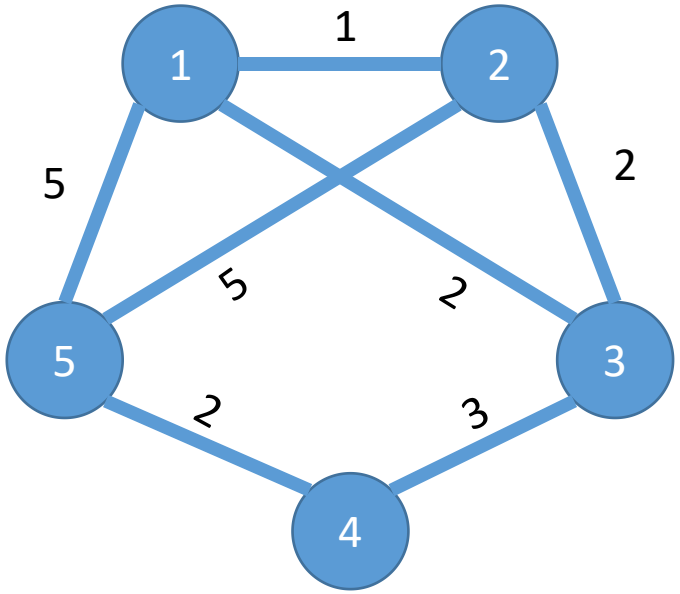


因为每次我们选的都是最小的边，所以最后的生成树一定是最小生成树，最小生成树也可能不唯一。

Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

假设目前输入的加权无向图为右图
此时有5个集合{ {1}，{2}，{3}，{4}，{5} }
生成树中没有边



此时的边有：
3 4 3
1 2 1
2 5 5
1 5 5
2 3 2
1 3 2
4 5 2

→

排序后：
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5

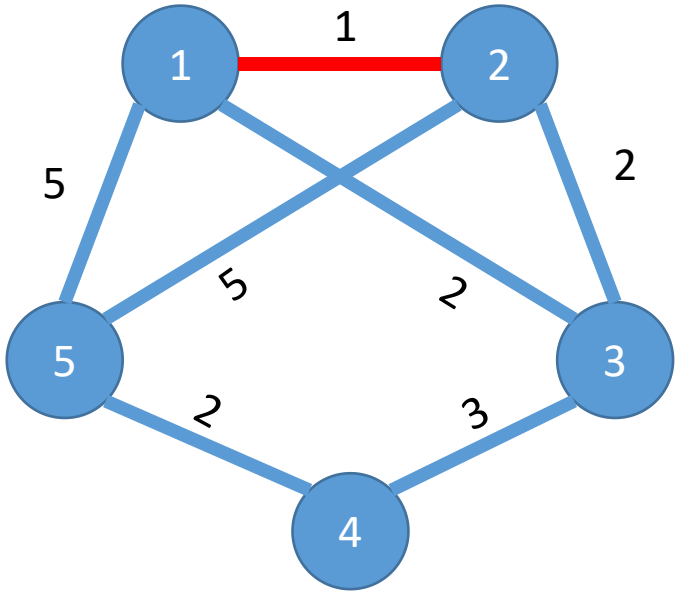


Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第一次选择的是<1，2>这条边，将这条边加入到生成树中，
并且将它的两个顶点1、2合并成一个集合。
此时有4个集合{ {1，2}，{3}，{4}，{5} }
生成树中有一条边{ <1,2> }

```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```



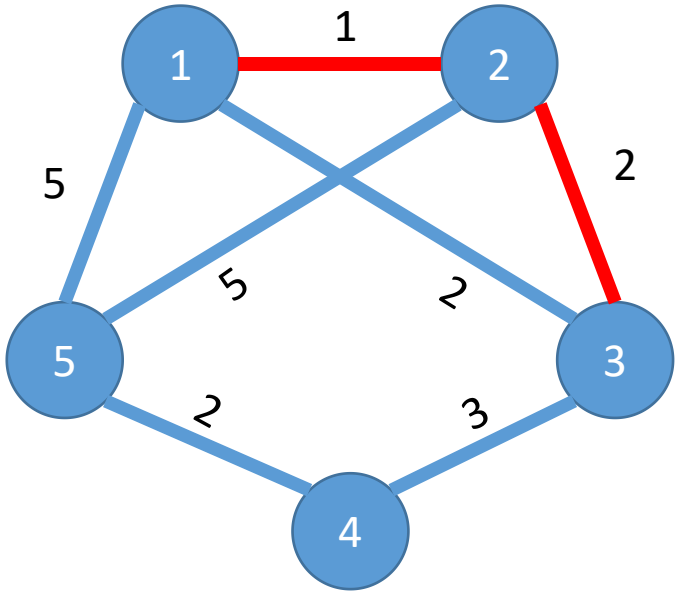
COOLMAKER 酷哥创客 人工智能教育

Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第二次选择的是<2,3>这条边，将这条边加入到生成树中，
并且将它的两个顶点2、3合并成一个集合。
此时有3个集合{ {1，2，3}，{4}，{5} }
生成树中有两条边{ <1,2>，<2,3> }

```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```

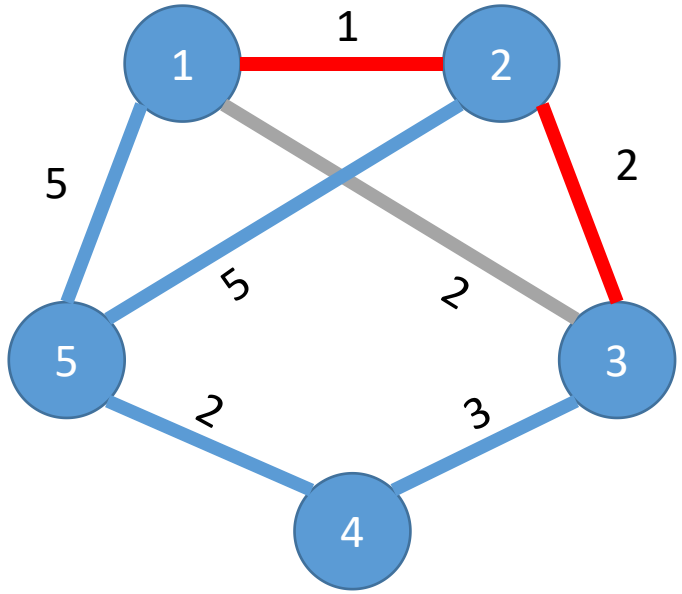


Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第三次选择的是<1,3>这条边，但是顶点1、3已经在同一个集合里了，所以这条边不需要加入生成树中。
此时有3个集合{ {1，2，3}，{4}，{5} }
生成树中有两条边{ <1,2>，<2,3> }

```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```

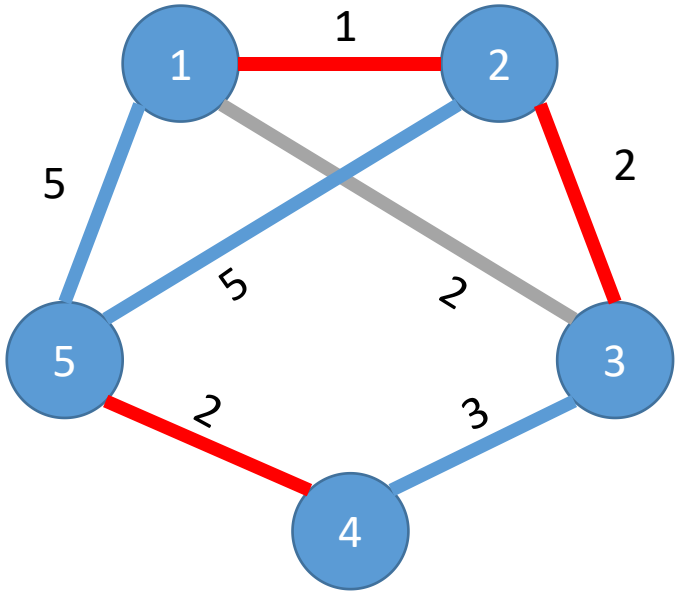


Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第四次选择的是<4,5>这条边，将这条边加入到生成树中，
并且将它的两个顶点4、5合并成一个集合。
此时有2个集合{ {1，2，3}，{4，5} }
生成树中有三条边{ <1,2>，<2,3> ，<4,5>}

```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```

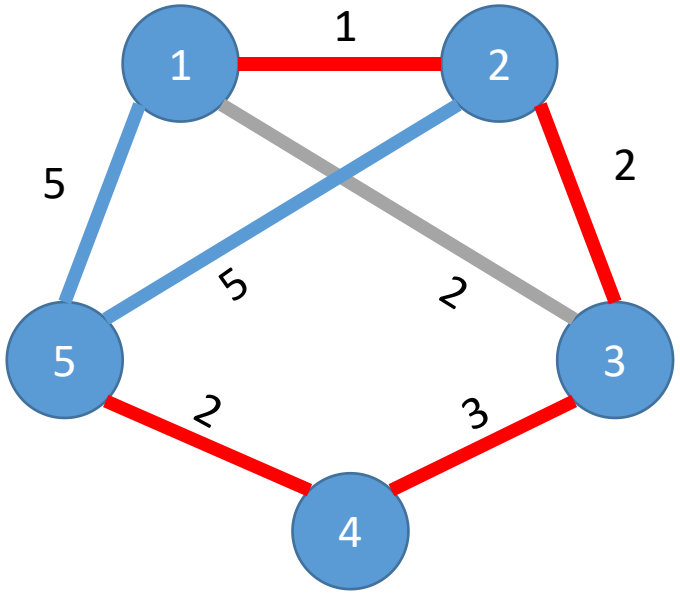


Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第五次选择的是<3,4>这条边，将这条边加入到生成树中，
并且将它的两个顶点3、4合并成一个集合。
此时有1个集合{ {1，2，3，4，5} }
生成树中有四条边{ <1,2>，<2,3>，<4,5>，<3,4>}

1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5

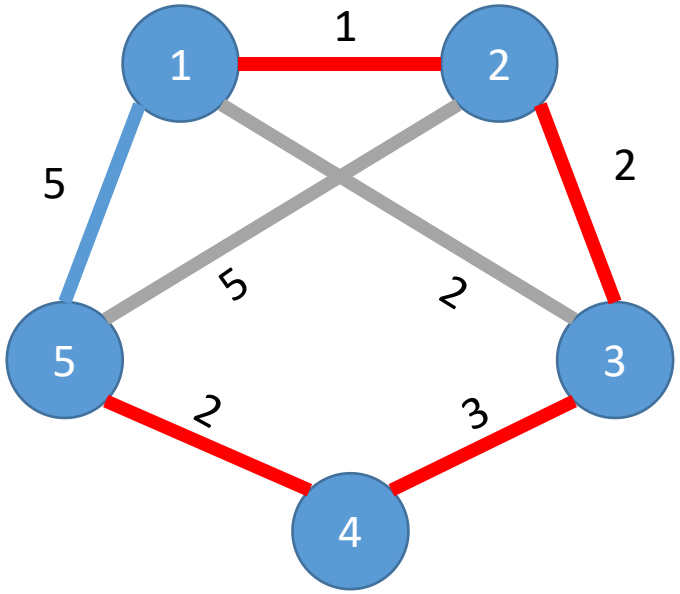


COOLMAKER 酷哥创客 人工智能教育

Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。
第六次选择的是<2,5>这条边，但是顶点2、5已经在同一个集合里了，而且已经选择了n-1条边了，所以这条边不需要加入生成树中。
此时有1个集合{ {1，2，3，4，5} }
生成树中有四条边{ <1,2>，<2,3>，<4,5>，<3,4>}

1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5

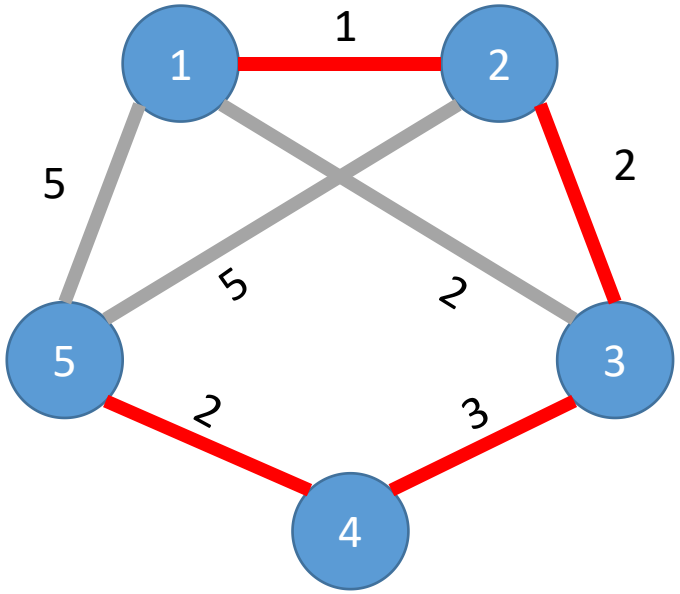


Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

第七次选择的是<1,5>这条边，但是顶点1、5已经在同一个集合里了，而且已经选择了n-1条边了，所以这条边不需要加入生成树中。
此时有1个集合{ {1，2，3，4，5} }
生成树中有四条边{ <1,2>，<2,3> ，<4,5>，<3,4>}

```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```

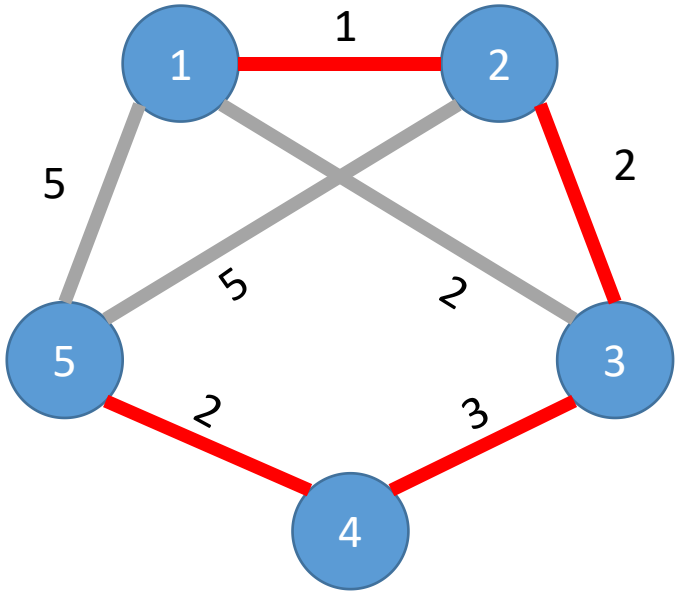


COOLMAKER 酷哥创客 人工智能教育

Kruskal算法的原理

Kruskal算法开始时，认为每一个点都是孤立的，分属于n个独立的集合。
Kruskal每次都选择一条最小的边，而且这条边的两个顶点分属于两个不同的集合。将选取的这条边加入最小生成树，并且合并集合。

1个集合{ {1，2，3，4，5} }
生成树中有四条边{ <1,2>，<2,3> ，<4,5>，<3,4>}
算法结束，最小生成树权值为8。



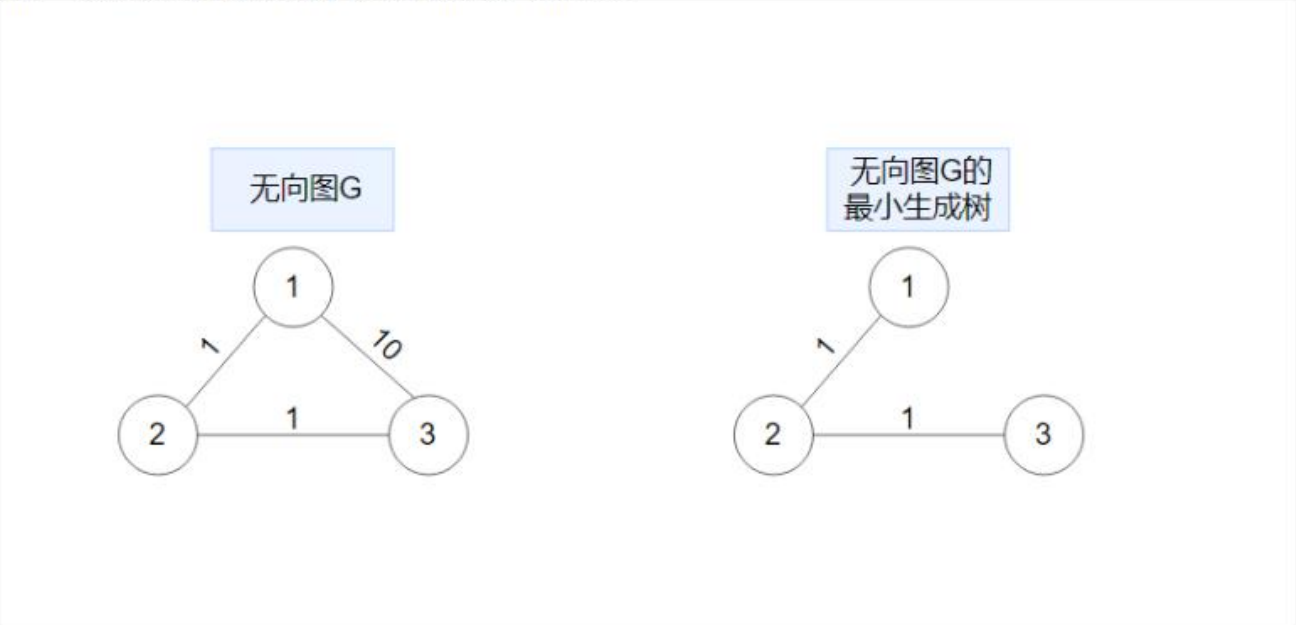


```
1 2 1
2 3 2
1 3 2
4 5 2
3 4 3
2 5 5
1 5 5
```

3047: 【模板】最小生成树Kruskal

时间限制: 1 Sec 内存限制: 128 MB
提交: 0 解决: 4
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

题目描述
这是一道最小生成树Kruskal的模板题,本题与【模板】最小生成树Prim，仅仅只有n和m的大小不同
给出一个无向图，求出最小生成树，如果该图不连通，则输出orz



输入
第一行输入2个正整数n，m，代表这个无向图有n个点，m条边。（1<=n<=1e5,1<=m<=2e5）
后面输入m行，每行有3个正整数 x，y，z，代表第i条边所连接的2个点和这条边的权值。(1<=x,y<=n, 1<=v<=10000)

输出
如果给的无向图是连通的，也就是能够求出一个最小生成树，请输出这个最小生成树的权值，否则请输出orz

样例输入
```
3 3
1 2 1
1 3 10
2 3 1
```

样例输出
```
2
```

提示
选择<1,2,1><2,3,1>这两条边此时可以生成一个最小生成树，此时权值也就是1+1=2

知识宝典

Kruskal算法的实现

sort

定义边的结构体
以及边的比较方法

```
struct node
{
    int x, y;
    int w;
};
bool cmp(node a,node b)
{
    return a.w<b.w;
}
```

并查集

```
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
```



```
//最小生成树Kruskal--sort版
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x, y;
    int w;
};
bool cmp(node a,node b)
{
    return a.w<b.w;
}
int n, m, k;
node q[200005];
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1; i<=m; i++)
    {
        cin>>q[i].x>>q[i].y>>q[i].w;
    }
    sort(q+1, q+m+1,cmp);
    for(int i=1; i<=m; i++)
    {
        node now=q[i];
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
int main()
{
    IO;
    solve();
    return 0;
}
```

知识宝典

Kruskal算法的实现

sort

```
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1; i<=m; i++)
    {
        cin>>q[i].x>>q[i].y>>q[i].w;
    }
    sort(q+1, q+m+1,cmp);
    for(int i=1; i<=m; i++)
    {
        node now=q[i];
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
```

1. 读入n，m
2. 读入m条边
3. 对m条边进行排序
4. 遍历m条边，如果这条边连接的2个点不在同一个集合，我们就把这条边加入最小生成树，同时把着2个集合合并

```
//最小生成树Kruskal--sort版
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x, y;
    int w;
};
bool cmp(node a,node b)
{
    return a.w<b.w;
}
int n, m, k;
node q[200005];
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1; i<=m; i++)
    {
        cin>>q[i].x>>q[i].y>>q[i].w;
    }
    sort(q+1, q+m+1,cmp);
    for(int i=1; i<=m; i++)
    {
        node now=q[i];
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
int main()
{
    IO;
    solve();
    return 0;
}
```

Kruskal算法的实现

优先队列

在讲Kruskal优先队列实现前，我们先了解一下优先队列吧。

- 普通的队列是一种先进先出的数据结构，元素在队列尾追加，而从队列头删除。在优先队列中，元素被赋予优先级。当访问元素时，具有最高优先级的元素最先删除的行为特征。

- 优先队列(priority_queue)，实际上，它的本质就是一个堆heap，利用一个大根堆(堆排序)完成。而C++STL中的优先队列(priority_queue)就是在这个队列的基础上，把其中的元素加以排序。其内部实现是一个二叉堆。所以优先队列其实就是把堆模板化，将所有入队的元素排成具有单调性的一队，方便我们调用。



Kruskal算法的实现

优先队列

优先队列在头文件#include <queue>中；
基本操作有：

- empty( ) //判断一个队列是否为空
- pop( ) //删除队顶元素
- top( ) //返回优先队列的队顶元素
- push( ) //加入一个元素
- size( ) //返回优先队列中拥有的元素个数



Kruskal算法的实现

优先队列

其声明格式为：priority_queue <Type>
priority_queue<Type, Container, Functional>

- Type 就是数据类型，Container 就是容器类型，Container必须是用数组实现的容器，比如vector,deque等等，但不能用 list。STL里面默认用的是vector， Functional 就是比较的方式
- 当需要用自定义的数据类型时才需要传入这三个参数，使用基本数据类型时，只需要传入数据类型，默认是大根堆(优先级高的在前)



COOLMAKER 酷哥创客 人工智能教育

Kruskal算法的实现

优先队列

因为大部分时候使用都是用自定义数据类型所以还需要了解优先队列(priority_queue)的一般常用自定义比较方法

- 重载<运算符：

```
struct node
{
    int x;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
```

- 自定义友函数：

```
struct node {
    int x,w;
    friend bool operator <(node p,node q) {
        return p.w>q.w;
    }
};
```

他们的声明格式都是 priority_queue<node> q;



由于是大根堆默认从大到小，所以比较方法里面<号要写成>号，修改什么是“大”从而实现从小到大

知识宝典

Kruskal算法的实现

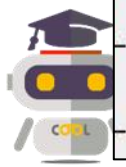
优先队列

定义边的结构体
以及结构体在优先队列里的比较方法，
优先队列是大根堆，
也就是从大到小，所以要w>a.w，更改大小的定义

```
struct node
{
    int x, y;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
```

并查集

```
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
```




```
//最小生成树Kruskal--优先队列
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x, y;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
int n, m, k;
priority_queue<node>q;
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1, x, y, w; i<=m; i++)
    {
        cin>>x>>y>>w;
        q.push({x, y, w}); //用优先队列排序，或者放数组里用sort排序
    }
    while(!q.empty())
    {
        node now=q.top();
        q.pop();
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            if(now.x==1||now.y==1)cout<<now.w<<" ";
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
int main()
{
    IO;
    solve();
    return 0;
}
```

知识宝典

Kruskal算法的实现

优先队列

```
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1, x, y, w; i<=m; i++)
    {
        cin>>x>>y>>w;
        q.push({x, y, w}); //用优先队列排序，或者放数组里用sort排序
    }
    while(!q.empty())
    {
        node now=q.top();
        q.pop();
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            if(now.x==1||now.y==1)cout<<now.w<<" ";
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
```

1. 读入n，m
2. 读入m条边，并加入优先队列里
3. 遍历m条边，如果这条边连接的2个点不在同一个集合，我们就把这条边加入最小生成树，同时把着2个集合合并

```
//最小生成树Kruskal--优先队列
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x, y;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
int n, m, k;
priority_queue<node>q;
int pa[100005];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a, int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
void solve()
{
    cin>>n>>m;
    int ans=0, cnt=0;
    for(int i=1; i<=n; i++)pa[i]=i;
    for(int i=1, x, y, w; i<=m; i++)
    {
        cin>>x>>y>>w;
        q.push({x, y, w}); //用优先队列排序，或者放数组里用sort排序
    }
    while(!q.empty())
    {
        node now=q.top();
        q.pop();
        if(merge_set(now.x, now.y)) //看看是不是再同一个集合里，不是就连边
        {
            cnt++;
            if(now.x==1||now.y==1)cout<<now.w<<" ";
            ans+=now.w;
        }
    }
    if(cnt!=n-1)//一共要连n-1条边，否则肯定不是连通图
    {
        cout<<"orz";
    }
    else
    {
        cout<<ans;
    }
}
int main()
{
    IO;
    solve();
    return 0;
}

```

习题训练

COOLMAKER 酷哥创客 人工智能教育

1327: 局域网

时间限制: 1 Sec 内存限制: 128 MB
提交: 7 解决: 6
[提交] [状态] [讨论版] [命题人: admin] [Edit] [TestData]

题目描述
某个局域网内有n(n≤100)台计算机，由于搭建局域网时工作人员的疏忽，现在局域网内的连接形成了回路，我们知道如果局域网形成回路那么数据将不停的在回路内传输，造成网络卡的现象。因为连接计算机的网线本身不同，所以有一些连线不是很畅通，我们用f(i,j)表示i,j之间连接的畅通程度(f(i,j)≤1000)，f(i,j)值越小表示i,j之间连接越通畅，f(i,j)为0表示i,j之间无网线连接。现在我们需要解决回路问题，我们将除去一些连线，使得网络中没有回路，并且被除去网线的Σf(i,j)最大，请求出这个最大值。

输入
第一行两个正整数n k

接下来的k行每行三个正整数i j m表示i,j两台计算机之间有网线联通，通畅程度为m。

输出
一个正整数，Σf(i,j)的最大值。

样例输入
```
5 5
1 2 8
1 3 1
1 5 3
2 4 5
3 4 2
```

样例输出
```
8
```

提示
无

来源/分类
第四章图论算法 第五节最小生成树

[提交] [状态] [Edit] [TestData]

求如何删去更多权值更大的边，还要保证没有回路，且任意2个计算机能够直接或者间接的连接。
也就是说删完之后的图应该权值和尽可能小，还要全连通，无回路；这不就是要我们求出这个加权连通图的最小生成树了嘛，最后总的权值和减去最小生成树的权值不就是我们要除去的网线权值和了嘛

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x,y;
    int w;
    bool operator <(const node &a)const
{
    return w>a.w;
}
};
int n,m,k;
priority_queue<node>q;
int pa[100005];
int s[400][400];
int find_set(int x)
{
    if(pa[x]==x)return x;
    pa[x]=find_set(pa[x]);
    return pa[x];
}
bool merge_set(int a,int b)
{
    int aa=find_set(a);
    int bb=find_set(b);
    if(aa!=bb)
    {
        pa[aa]=bb;
        return 1;
    }
    return 0;
}
void solve()
{
    cin>>n>>m;
    int ans=0,sum=0;
    for(int i=1;i<=n;i++)pa[i]=i;
    for(int i=1,x,y,w;i<=m;i++)
    {
        cin>>x>>y>>w;
        q.push({x,y,w});
        sum+=w;
    }
    while(!q.empty())
    {
        node now=q.top();q.pop();
        if(merge_set(now.x,now.y))
        {
            ans+=now.w;
        }
    }
    cout<<sum-ans;
}
int main()
{
    IO;
    solve();
    return 0;
}
```



最小生成树Prim算法





Prim算法的原理

Prim算法的核心思想是：

从一个任意的点开始，逐渐扩张出一个连通图，依次增加与该连通图相邻的权重最小的点，直到所有点都加入到图中，此时就得到了最小生成树。因此，Prim算法依靠扩张连通图和选择最小值的操作来寻找最小生成树。



Prim算法的原理

具体步骤如下：

- 随机选择任意一个节点作为起始节点，将该节点加入到最小生成树(MST)集合中。
- 遍历最小生成树(MST)集合中已有的所有节点，找到与之相邻、且不在最小生成树(MST)集合中的权值最小的边，将相应节点加入到最小生成树(MST)集合中。
- 重复上述操作直到最小生成树(MST)集合中包含所有节点。

Prim算法的时间复杂度为O(n^2)，可以使用堆来优化选择最小值的操作，提高算法的效率。



Prim算法的原理

下面我们来模拟一下这个过程吧！！！

- 简单来说Prim算法，先随机选择一个蓝点变成白点，作为初始点；
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的；
- 直到把所有蓝点变成白点。



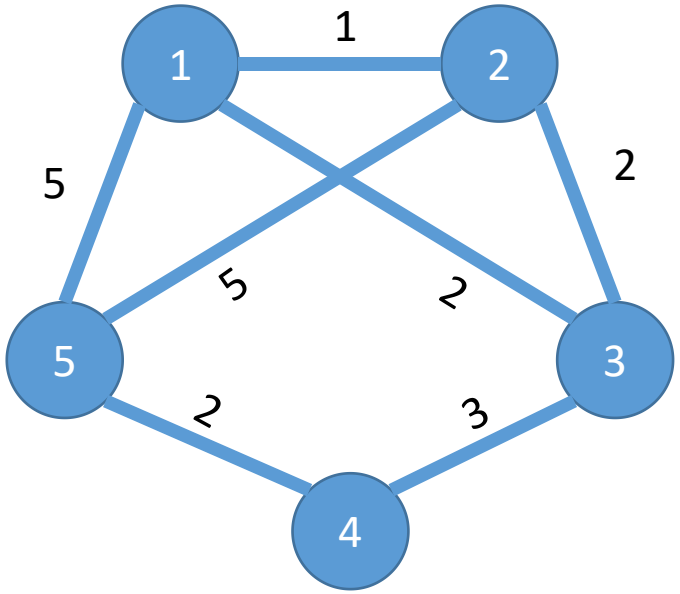
COOLMAKER 酷哥创客 人工智能教育

Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

假设目前输入的加权无向图为右图
此时有0个白点{}，5个蓝点{1，2，3，4，5}
此时各个点的权值：

点	1	2	3	4	5
点权	∞	∞	∞	∞	∞

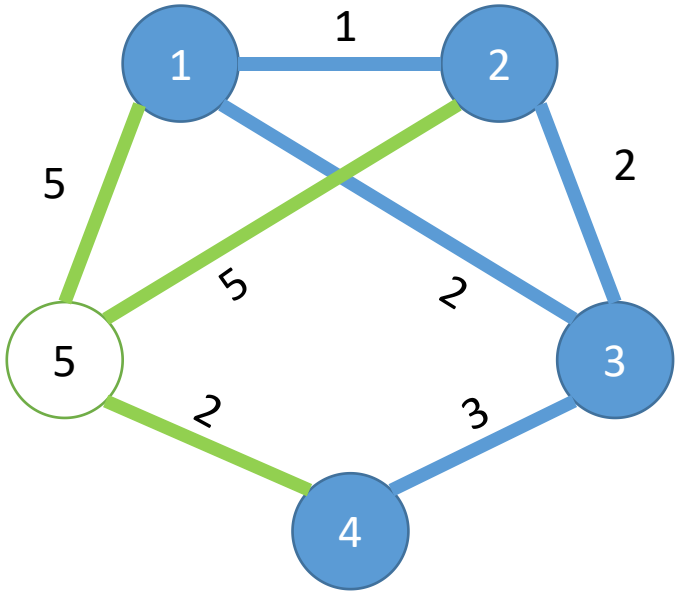


Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

我们先随意选择一个蓝点变成白点，比如 5，然后更新附近蓝点的最小点权
此时有1个白点{5}，4个蓝点{1，2，3，4}
此时各个点的权值：

点	1	2	3	4	5
点权	5	5	∞	2	0

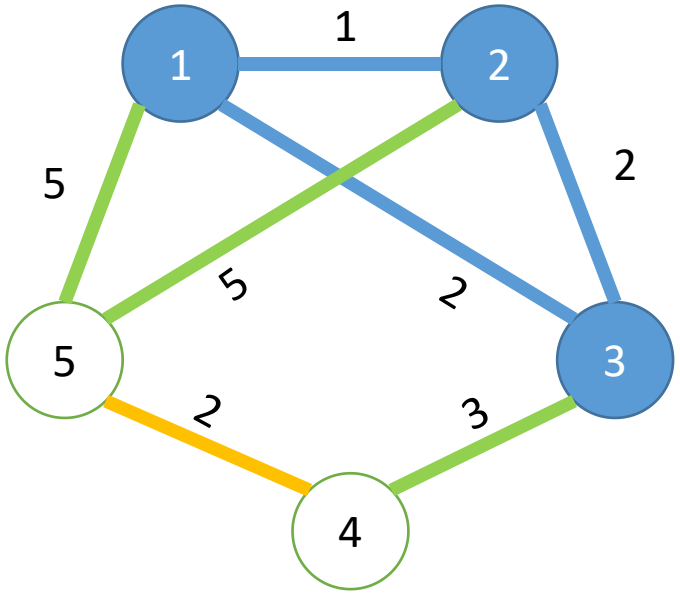


Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

然后目前选择最小点权的蓝点4变成白点，然后更新附近蓝点的最小点权
此时有2个白点{4，5}，3个蓝点{1，2，3}
此时各个点的权值：

点	1	2	3	4	5
点权	5	5	3	2	0



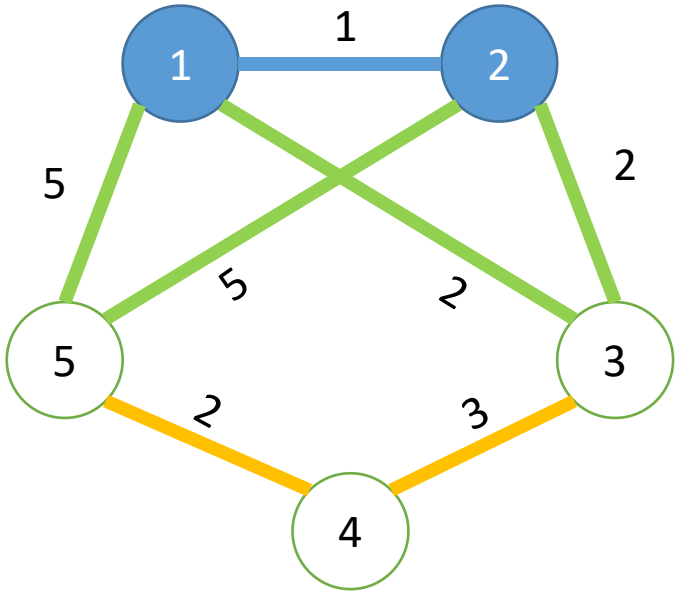
COOLMAKER 酷哥创客 人工智能教育

Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

然后目前选择最小点权的蓝点3变成白点，然后更新附近蓝点的最小点权
此时有3个白点{3，4，5}，2个蓝点{1，2}
此时各个点的权值：

点	1	2	3	4	5
点权	2	2	3	2	0



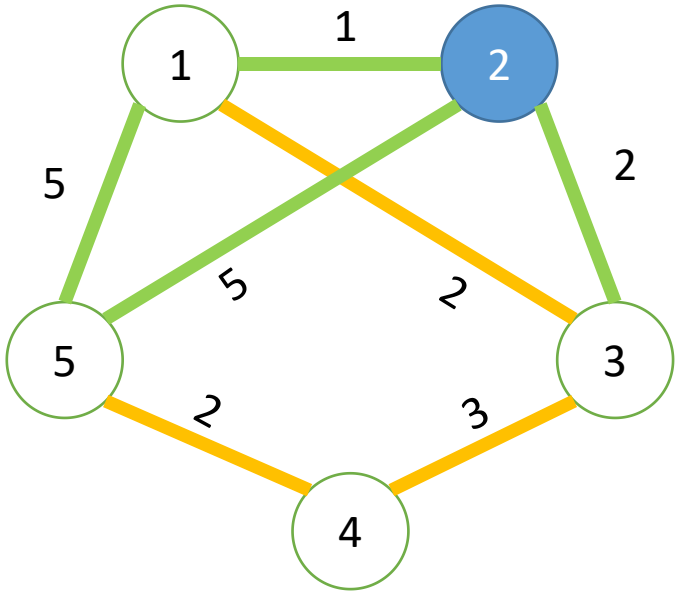
COOLMAKER 酷哥创客 人工智能教育

Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

然后目前选择最小点权的蓝点1变成白点，然后更新附近蓝点的最小点权
此时有4个白点{1，3，4，5}，1个蓝点{1}
此时各个点的权值：

点	1	2	3	4	5
点权	2	1	3	2	0

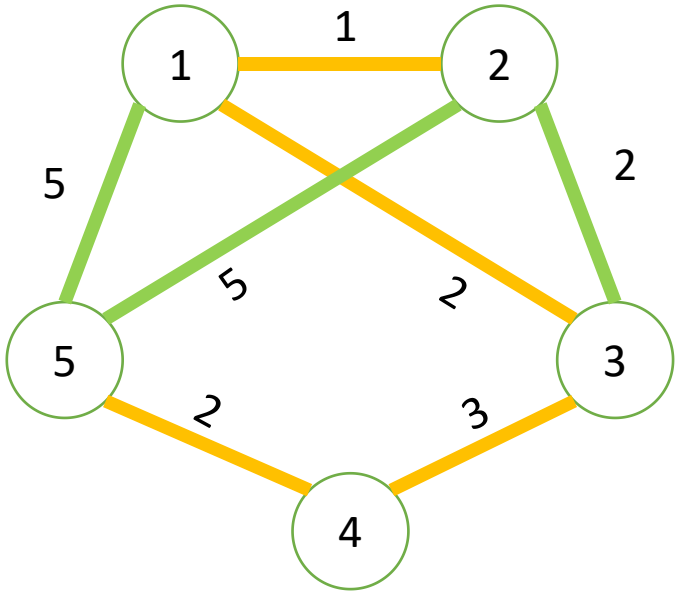


Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

然后目前选择最小点权的蓝点2变成白点，然后更新附近蓝点的最小点权
此时有5个白点{1，2，3，4，5}，0个蓝点{}
此时各个点的权值：

点	1	2	3	4	5
点权	2	1	3	2	0

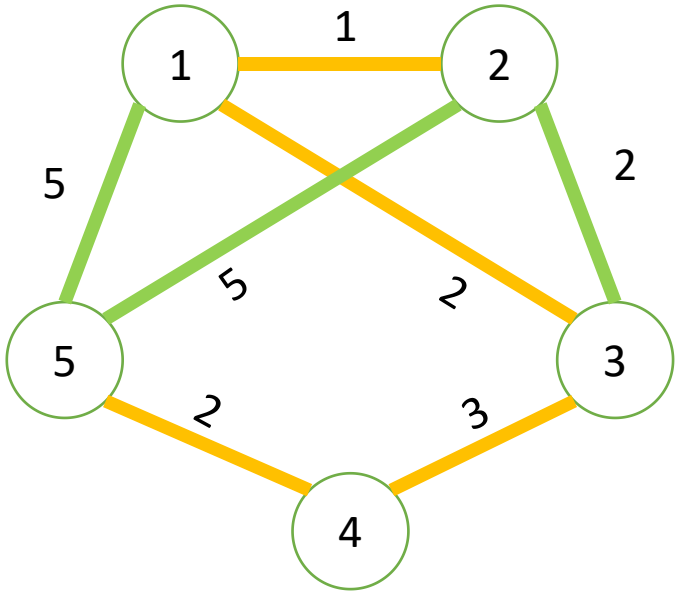


Prim算法的原理

- 简单来说Prim算法,先随机选择一个蓝点变成白点;
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的;
- 直到把所有蓝点变成白点。

现在所有的蓝点都变成白点了，所有点现在的点权相加就是最小生成树的权值了

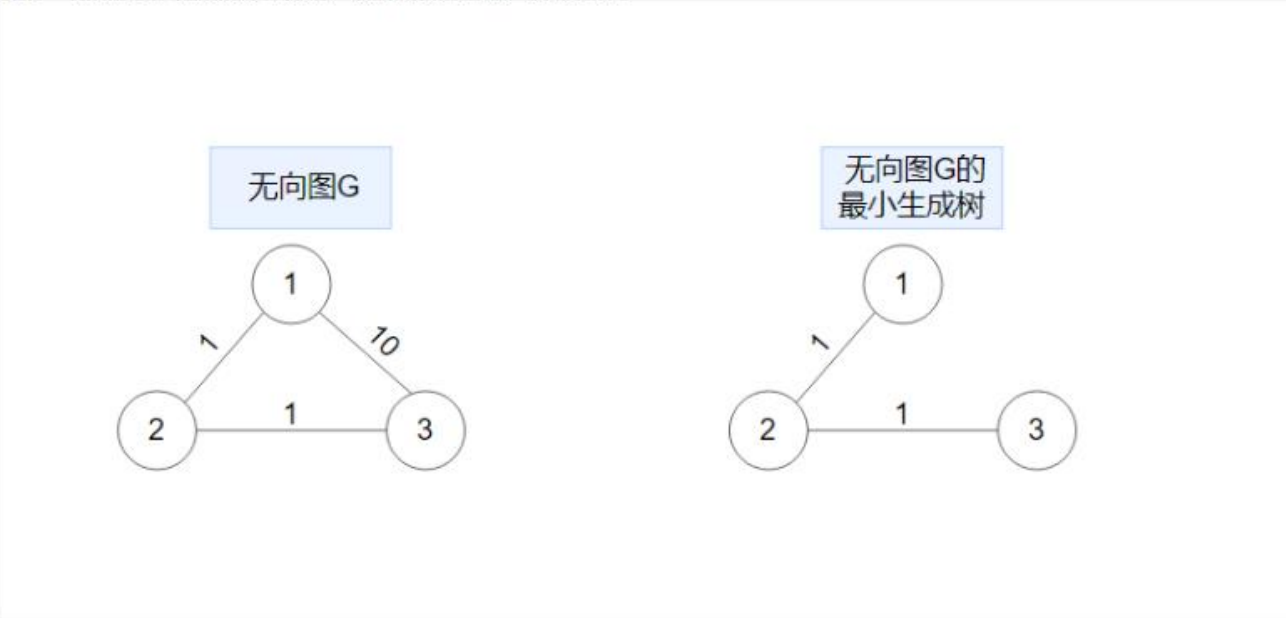
点	1	2	3	4	5
点权	2	1	3	2	0



3048: 【模板】最小生成树Prim

时间限制: 2 Sec 内存限制: 128 MB
提交: 0 解决: 3
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

题目描述
这是一道最小生成树Prim的模板题，本题与【模板】最小生成树Kruskal，仅仅只有n和m的大小不同
给出一个无向图，求出最小生成树，如果该图不连通，则输出orz



输入
第一行输入2个正整数n，m，代表这个无向图有n个点，m条边。（1<=n<=1e3,1<=m<=2e6）
后面输入m行，每行有3个正整数 x，y，z，代表第i条边所连接的2个点和这条边的权值。(1<=x,y<=n, 1<=v<=10000)

输出
如果给的无向图是连通的，也就是能够求出一个最小生成树，请输出这个最小生成树的权值，否则请输出orz

样例输入
```
3 3
1 2 1
1 3 10
2 3 1
```

样例输出
```
2
```

提示
选择<1,2,1><2,3,1>这两条边此时可以生成一个最小生成树，此时权值也就是1+1=2

C++ 知 识 宝 典

Prim算法的实现

Prim算法部分

- 先随机选择一个蓝点变成白点；
- 然后每次循环都将一个蓝点变为白点，并且此蓝点与白点相连的最小边权还是当前所有蓝点中最小的；
- 直到把所有蓝点变成白点。



```cpp
//最小生成树Prim--朴素
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

int n,m,k;
int v[1005][1005];
int dis[1005];
bool f[1005];
void solve()
{
    cin>>n>>m;
    int ans=0;
    //初始化每条边的值
    memset(v,0x3f,sizeof(v));
    for(int i=1,x,y,w;i<=m;i++)
    {
        cin>>x>>y>>w;
        if(v[x][y]>w)//更新每条边的最小值
        {
            v[x][y]=w;
            v[y][x]=w;
        }
    }
    //促使化每个点的点权
    memset(dis,0x3f,sizeof(dis));
    memset(f,0,sizeof(f));
    dis[1]=0;//起始点X
    for(int i=1;i<=n;i++)//一共要n次，每次找到一个权值最小的点
    {
        int x=-1;
        for(int j=1;j<=n;j++)//prim 算法每次从已经连通的点里，找到还未扩展的最小点，开始扩展
        {
            if(!f[j]&&dis[j]<=10000&&(x==-1||dis[j]<dis[x]))x=j;
        }
        if(x==-1)//不能扩展第i+1个点，说明图不连通
        {
            cout<<"orz";
            return ;
        }
        ans+=dis[x];//加入这个点的权值
        f[x]=1;
        for(int j=1;j<=n;j++)
        {
            if(!f[j])dis[j]=min(dis[j],v[x][j]);//如果是还未扩展的点，更新最小权值
        }
    }
    cout<<ans;
}
int main()
{
    IO;
    solve();
    return 0;
}
```

Prim算法堆优化

我们先来复习一下优先队列的用法吧！！！

每次找点都需要花费0(n)的时间找，那有没有可以优化这个过程的方法呢？

可以注意到每次都是要找到点权最小的点，来扩展，我们可以用上节课学过的可以自排序的优先队列来进行优化。



Prim算法堆优化

优先队列

优先队列在头文件#include <queue>中；
其声明格式 priority_queue <Type>
基本操作有：

- empty( ) //判断一个队列是否为空
- pop( ) //删除队顶元素
- top( ) //返回优先队列的队顶元素
- push( ) //加入一个元素
- size( ) //返回优先队列中拥有的元素个数



Prim算法堆优化

优先队列

因为大部分时候使用都是用自定义数据类型所以还需要了解优先队列(priority_queue)的一般常用自定义比较方法

- 重载<运算符：

```
struct node
{
    int x;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
```

- 自定义友函数：

```
struct node {
    int x,w;
    friend bool operator <(node p,node q) {
        return p.w>q.w;
    }
};
```

他们的声明格式都是 priority_queue<node> q;



由于是大根堆默认从大到小，所以比较方法里面<号要写成>号，修改什么是“大”从而实现从小到大

知识宝典

Prim算法堆优化

声明优先队列
以及重载比较方法

Prim算法部分

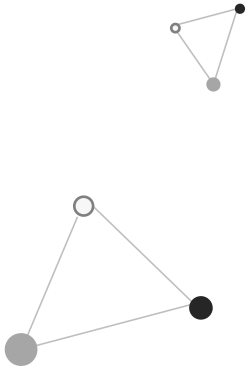
- 先随机选择一个蓝点变成白点；
- 然后每次循环都将一个蓝点u变为白点，并且此蓝点u与白点相连的最小边权还是当前所有蓝点中最小的；
- 直到把所有蓝点变成白点。

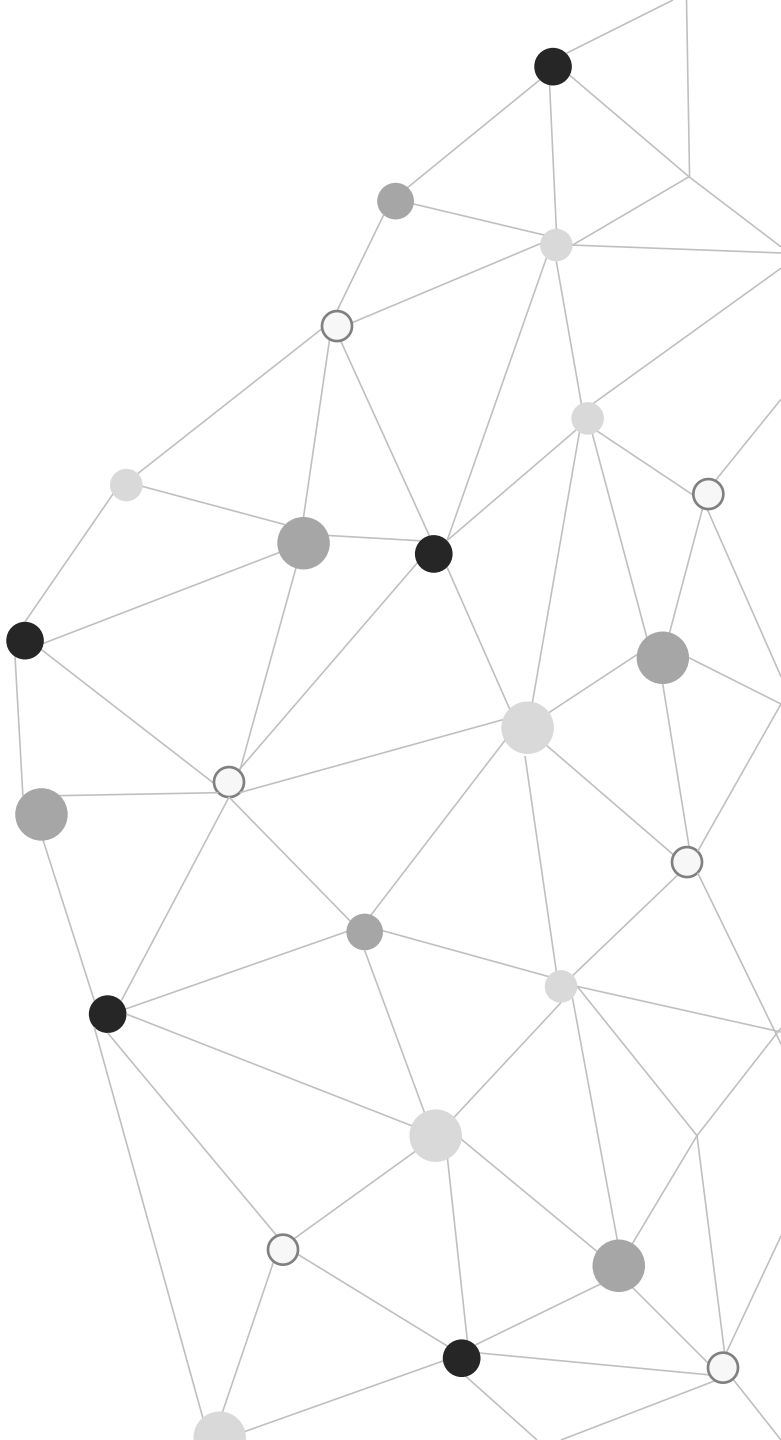
```cpp
//最小生成树Prim--堆优化
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;

struct node
{
    int x;
    int w;
    bool operator <(const node &a)const
    {
        return w>a.w;
    }
};
int n, m, k;
int v[1005][1005];
int dis[1005];
bool f[1005];
priority_queue<node>q;
void solve()
{
    cin>>n>>m;
    int ans=0,cnt=0;
    memset(v, 0x3f, sizeof(v));
    for(int i=1, x, y, w; i<=m; i++)
    {
        cin>>x>>y>>w;
        if(v[x][y]>w)
        {
            v[x][y]=w;
            v[y][x]=w;
        }
    }
    memset(dis, 0x3f, sizeof(dis));
    memset(f, 0, sizeof(f));
    dis[1]=0;//起始点
    q.push({1,0});
    while(!q.empty())
    {
        node now=q.top();q.pop();
        if(f[now.x])continue;//已经扩展过了，就不继续扩展了
        f[now.x]=1;
        ans+=now.w;
        cnt++;
        for(int i=1;i<=n;i++)
        {
            if(!f[i]&&dis[i]>v[now.x][i])//如果是未被扩展的点更新最小权值
            {
                dis[i]=v[now.x][i];
                q.push({i,dis[i]});
            }
        }
    }
    if(cnt!=n)//如果没用扩展n个点说明图不连通
    {
        cout<<"orz";
    }else
    {
        cout<<ans;
    }
}
int main()
{
    IO;
    solve();
    return 0;
}
```



回 顾 总 结







COOLMAKER
酷哥创客 人工智能教育

酷哥创客

感谢您的观看





分层图思想





分层图与分层图思想

COOLMAKER
酷哥创客 人工智能教育

分层图，顾名思义就是分很多层的图,也就是类似二维数组,不再是一个单一的平面,而是一个立体化的东西,不只有长宽，也有了自己的厚度,即为层数

分层图思想。该思想通过挖掘问题性质，将原问题抽象得出的图复制为若干层并连接形成更大的图，使本来难以用数学语言表达的图论模型变得简明严谨，为进一步解决问题打下了良好的基础。

